

F25-038-D-QCanvas: A Unified Quantum Circuit Simulation and Education Platform

Project Team

Umer Farooq	22I-0891
Hussan Waseem Syed	22I-0893
Muhammad Irtaza Khan	22I-0911

Session 2022-2026

Supervised by

Dr. Imran Ashraf

Co-Supervised by

Dr. Muhammad Nouman Noor



Department of Computer Science

**National University of Computer and Emerging Sciences
Islamabad, Pakistan**

May, 2026

Contents

Acknowledgements	1
Abstract	2
1 Introduction	3
1.1 Existing Solutions	4
1.2 Problem Statement	5
1.3 Scope	5
1.3.1 In Scope	6
1.3.2 Out of Scope	6
1.4 Modules	7
1.4.1 Conversion Module	7
1.4.2 Simulation Module (QSim SDK)	7
1.4.3 Validation Module	8
1.4.4 Web IDE Module	8
1.4.5 Educational Intelligence Module	8
1.4.6 Benchmarking Module	9
1.5 Work Division	10
1.6 User Classes and Characteristics	11
1.7 Assumptions, Dependencies, and Constraints	11
1.7.1 Assumptions	11
1.7.2 Dependencies	11
1.7.3 Constraints	12
2 Project Requirements	13
2.1 Functional Requirements	13
2.1.1 Conversion Module	13
2.1.2 Simulation Module (QSim SDK)	14
2.1.3 Hybrid CPU–QPU Module	14
2.1.4 Web IDE Module	15
2.1.5 Educational Intelligence Module	15
2.1.6 API and Realtime	15
2.1.7 Validation	16

2.2	Non-Functional Requirements	16
2.2.1	Reliability	16
2.2.2	Usability	16
2.2.3	Performance	17
2.2.4	Security	17
2.3	External Interface Requirements	18
2.3.1	User Interfaces	18
2.3.1.1	Web IDE Interface	18
2.3.1.2	Results and Visualisation	18
2.3.2	Software Interfaces	18
2.3.2.1	Quantum Framework APIs	18
2.3.3	Communication Interfaces	18
2.3.3.1	REST API	18
2.3.3.2	WebSocket Interface	19
2.4	Use Cases	20
2.4.1	Functional Requirements Traceability	20
2.4.2	UC-01: Launch Web IDE	20
2.4.3	UC-02: Write Circuit Code	20
2.4.4	UC-03: Convert to OpenQASM	21
2.4.5	UC-04: Execute Circuit	21
2.4.6	UC-05: Visualise Results	22
2.4.7	UC-06: Handle Validation Errors	22
2.4.8	UC-07: Export OpenQASM	23
2.4.9	UC-08: Load Example Circuit	23
2.4.10	UC-09: Query Backend Capabilities	24
2.4.11	UC-10: Hybrid CPU–QPU Execution	24
2.4.12	UC-11: Hover Explanation (Quantum Explain It)	25
2.4.13	UC-12: Earn Achievement	25
2.4.14	UC-13: Share Circuit Recipe	25
2.5	Use Cases Diagram	27
2.6	Domain Model Diagram	28
3	System Overview	29
3.1	Architectural Design	29
3.1.1	Layered Responsibilities	30
3.1.2	Technology Stack	30
3.1.2.1	Frontend Technologies	31
3.1.2.2	Backend Technologies	31
3.1.2.3	Quantum Computing Stack	31
3.1.3	Hybrid CPU–QPU Execution Model	32

3.1.3.1	Orchestration Boundary	32
3.1.3.2	Data Flow	32
3.2	Design Models	32
3.2.1	Activity Diagrams	32
3.2.1.1	Activity Diagram: Circuit Conversion and Execution Pipeline	32
3.2.1.2	Activity Diagram: User Registration and Session Issuance	33
3.2.2	Sequence Diagrams	34
3.2.2.1	Sequence Diagram: Unified Execution Workflow	34
3.2.2.2	Sequence Diagram: JWT Authentication Handshake	35
3.2.2.3	System Sequence Diagrams (SSD)	37
3.2.3	Architecture Diagram	40
3.2.4	Component Diagram	41
3.2.5	Package Diagram	42
3.2.6	Deployment Diagram	43
3.2.7	State Diagrams	44
3.2.7.1	State Diagram: Conversion Process	44
3.2.7.2	State Diagram: Simulation Job	45
3.2.8	Data Flow Diagrams (DFD)	46
3.2.8.1	DFD Level 0 (Context Diagram)	46
3.2.8.2	DFD Level 1	46
3.3	Data Design	47
3.3.1	Core Entities	47
3.3.2	Entity Relationship Diagram	48
3.3.3	Data Organisation and Processing	48
3.3.4	API Data Models	49
3.3.4.1	Request Models	49
3.3.4.2	Response Models	49
4	Implementation and Testing	50
4.1	Algorithm Design	50
4.2	External APIs and SDKs	51
4.3	Iteration-by-Iteration Implementation Progress	51
4.3.1	Iteration 1: Core Infrastructure and Basic Features	51
4.3.2	Iteration 2: Advanced Features and Hybrid Execution	51
4.3.3	Iteration 3: Educational Intelligence and Community Features	52
4.3.4	Iteration 4: QSim SDK Integration, Benchmarking, and SDK Packaging	52
4.4	User Interface Implementation	52
4.5	Testing and Validation	54

4.5.1	Testing Strategy	54
4.5.2	Unit Testing	54
4.5.3	Test Coverage	55
4.6	Evaluation and Benchmarking	55
4.6.1	Benchmark Setup	55
4.6.2	Compilation Time Results	56
4.6.3	Structural Benchmark Results	57
4.6.4	Scaling Behaviour Analysis	58
4.6.5	Statistical Equivalence Analysis	59
4.6.6	Quantum Volume Analysis	61
4.6.7	Holistic QPack Scores	61
4.6.8	Performance Evaluation	62
4.7	Non-Functional Requirements Testing and Validation Results	63
4.7.1	Reliability Validation: NFR-R1	63
4.7.2	Usability Validation: NFR-U3	63
4.7.3	Performance Validation: NFR-P1, NFR-P2, and NFR-P3	64
4.7.4	Security Validation: NFR-S1	64
4.7.5	Layout and Readability Cleanup Checklist	64
5	Conclusions and Future Work	66
5.1	Conclusion	66
5.2	Future Work	67
5.2.1	Near-Term Engineering Goals	67
5.2.1.1	Hardware Backend Integration	67
5.2.1.2	Noise-Aware Simulation	68
5.2.1.3	Expanded Framework Support	68
5.2.1.4	Production Infrastructure	68
5.2.2	Long-Term Research Directions	68
5.2.2.1	Advanced Visualisation and Analytics	68
5.2.2.2	Community and Research Collaboration	68
	Appendix	70
	References	72

List of Figures

2.1	Use Case Diagram	27
2.2	Domain Model	28
3.1	Simplified Box-and-Line Architecture of QCanvas	30
3.2	Activity Diagram — Circuit Conversion and Execution Pipeline	33
3.3	Activity Diagram — User Registration and Session Issuance	34
3.4	Sequence Diagram — Unified Execution Workflow	35
3.5	Sequence Diagram — JWT Authentication Handshake	36
3.6	SSD — Authentication Handler	37
3.7	SSD — Circuit OpenQASM Compilation	38
3.8	SSD — QSim Backend Simulation Execution	39
3.9	System Architecture Diagram	40
3.10	Component Diagram	41
3.11	Package Diagram	42
3.12	Deployment Diagram	43
3.13	State Diagram — Conversion	44
3.14	State Diagram — Simulation Job	45
3.15	DFD Level 0	46
3.16	DFD Level 1	46
3.17	Entity Relationship Diagram	48
4.1	QCanvas Home Page Interface	53
4.2	QCanvas Login Interface	53
4.3	QCanvas Web IDE Interface	54
4.4	Gate Count Comparison Across Frameworks	57
4.5	Circuit Depth Comparison Across Frameworks	58
4.6	GHZ State Depth Scaling Across Frameworks	59
4.7	Grover’s Algorithm Depth Scaling — PennyLane Super-Linear vs. Qiskit/Cirq Constant	59
4.8	Jensen–Shannon Divergence Heatmap (Pairwise Framework Equivalence)	60
4.9	QPack Radar Chart — Framework Holistic Score Comparison	62

List of Tables

1.1	Comparison of Existing Solutions	4
1.2	Work Division	10
1.3	User Classes and Characteristics	11
2.1	QCanvas Event-Response Matrix	19
2.2	Traceability between FRs and Use Cases	20
3.1	Core Data Entities and Key Attributes	47
3.2	Data Organisation and Processing	49
4.1	External APIs and SDKs Used	51
4.2	QCanvas Comprehensive Test Coverage (Huge Matrix Scaling Mode) . .	55
4.3	Mean Compilation Latency (ms) – Representative Algorithms at 8 Qubits	56
4.4	Structural Benchmark Summary (Selected Algorithms)	57
4.5	Scaling Behaviour (Power-Law Fit: Circuit Depth vs. Qubits)	58
4.6	Statistical Equivalence Summary (JSD, 4096 Shots)	60
4.7	Illustrative Effective Quantum Volume Comparison under Ideal Simulation	61
4.8	QPack Holistic Benchmark Scores	61
4.9	Non-Functional Requirements Validation Summary	63
1	Compilation Latency Summary – All Algorithms at Maximum Qubit Count	71

Acknowledgements

We express our sincere gratitude to our supervisor, Dr. Imran Ashraf, for his continuous guidance, valuable feedback, and encouragement throughout this Final Year Project. His direction helped us refine both the technical depth and the research focus of QCanvas and QSim. We are equally thankful to our co-supervisor, Dr. Muhammad Nouman Noor, for his support, insightful suggestions, and constructive reviews during the development and evaluation phases of the project.

We also thank our teammates for their commitment, collaboration, and consistency across all four iterations of the project. The successful completion of this work was only possible through shared effort, open communication, and mutual support.

We are deeply grateful to our families for their patience, motivation, and unwavering support during our undergraduate studies. Their trust and encouragement enabled us to stay focused through demanding academic and development milestones.

Finally, we acknowledge the Department of Computer Science at FAST National University of Computer and Emerging Sciences, Islamabad, for providing the academic environment and resources that made this project possible.

Abstract

The fragmentation of the quantum computing software ecosystem across competing frameworks—Qiskit, Cirq, and PennyLane—presents a significant barrier to educational accessibility and cross-framework research reproducibility. This project presents **QCanvas**, a unified quantum circuit simulation and education platform, and **QSim**, its companion simulation SDK, both packaged as installable Python tools. QCanvas employs OpenQASM 3.0 as a common intermediate representation, enabling framework-agnostic circuit compilation, validation, and execution through a FastAPI backend and a Monaco-based Web IDE. A comprehensive cross-framework benchmarking study across 12 standard quantum algorithms confirms that Qiskit and Cirq produce structurally equivalent, constant-depth circuits (QPack score ≈ 58.8 each), while PennyLane exhibits super-linear circuit growth for algorithms like Grover’s (QPack = 9.3), with Jensen–Shannon divergence near zero between Qiskit and Cirq output distributions. The platform additionally delivers an educational layer comprising hover-based gate explanations, a gamification engine with 44 unique achievements, and a community circuit-sharing library. An automated suite of 304 passing tests, supplemented by manual validation of selected user-facing features, supports the system’s correctness and reliability across all supported frameworks and backends.

CHAPTER 1

INTRODUCTION

QCanvas and its companion simulation library QSim represent the complete software deliverable of this Final Year Project. Both packages are fully developed, rigorously tested, and packaged for distribution as installable Python tools. This report documents the entire development lifecycle across four iterative project phases—from a foundational conversion prototype to a production-ready, community-accessible quantum computing platform.

The broader context motivating QCanvas is the persistent fragmentation of the quantum software ecosystem. As hardware vendors such as IBM, Google, and Rigetti advance NISQ-era devices [7], the accompanying software landscape has grown without standardisation. Researchers and students face a “Tower of Babel” scenario in which Qiskit, Cirq, and PennyLane each enforce unique syntax, compilation targets, and simulation APIs [1, 4, 8]. The absence of a unified intermediate representation in educational tooling obscures the critical translation step where high-level Python code becomes executable quantum instructions.

QCanvas resolves this by centering its architecture on OpenQASM 3.0 [3] as the universal intermediate representation. Every circuit—regardless of source framework—is compiled to this standard before execution, enabling transparent, consistent, and comparable simulation. The QSim SDK [10] provides the execution kernel, operating independently of the Web IDE and directly installable for programmatic use in research scripts and Jupyter notebooks.

This report presents the architecture, requirements, system design, benchmarking results, and technical achievements of the completed project, concluding with the Iteration 4 integration milestone that packaged both SDKs for distribution.

1.1 Existing Solutions

The quantum software landscape currently offers several frameworks and platforms, each addressing different aspects of the development workflow but none providing a unified, educational, and cross-framework simulation environment. Table 1.1 summarises key existing solutions and their limitations.

Table 1.1: Comparison of Existing Solutions

System	Overview	Limitations
IBM Quantum Experience	Cloud-based access to real quantum hardware and simulators via Qiskit. Provides circuit composition and execution.	Locked to IBM's Qiskit ecosystem. No cross-framework support. Requires internet access and an IBM account. No offline or SDK-level simulation.
Google Cirq / Colab	Google's framework for writing circuits targeting superconducting processors, typically used in Jupyter environments.	Cirq-only; no support for Qiskit or PennyLane. No unified IDE. Circuit visualisation is text-based ASCII.
Quirk	Browser-based drag-and-drop quantum circuit simulator for education.	No code-based interaction. Cannot parse real-world quantum programs. No OpenQASM support. Not extensible.
Quantum Katas (Microsoft)	Interactive Q# learning exercises.	Tied exclusively to Q# and the Microsoft ecosystem. Incompatible with Python-based frameworks used in research.
t ket> (TKET)	High-performance, retargetable compiler for NISQ devices supporting multiple backends [9].	Compilation-focused; no educational IDE. No beginner-friendly feedback. Not accessible as a unified web platform.

1.2 Problem Statement

Despite rapid progress in NISQ-era hardware and software, there is no unified simulation platform that allows quantum circuits from major frameworks (Qiskit, Cirq, PennyLane) to be expressed, compared, and executed through a single, standards-based workflow. Learners and researchers are forced to navigate duplicated concepts, incompatible APIs, and fragmented tooling across frameworks and simulators.

Specifically, the current ecosystem lacks:

- **A unified intermediate representation in tooling:** Most frameworks hide or only partially expose compiled quantum instructions, making it difficult to reason about what actually runs on the backend.
- **A single, framework-agnostic IDE:** There is no web-based environment where users can work with Cirq, Qiskit, and PennyLane side by side while seeing the same OpenQASM 3.0 view of their circuits.
- **A consistent simulation interface:** Users must switch between different simulators, configuration styles, and result formats when moving across frameworks.
- **Beginner-friendly feedback:** Existing professional tools prioritise flexibility over clarity, leaving students without guided error reporting, hover explanations, or visual circuit diagrams.
- **A programmatically accessible simulation SDK:** No lightweight Python package allows developers to import a quantum simulator directly into scripts without heavyweight dependencies.

QCanvas and QSim together address this problem by providing a unified interface where users write in their preferred framework, understand and execute their logic through the OpenQASM 3.0 lens, and access the underlying simulation engine as an installable SDK.

1.3 Scope

The scope of QCanvas is defined to deliver a functional, robust simulation and education environment across four development iterations.

1.3.1 In Scope

The system was designed and built to include the following capabilities:

- **Conversion Engine:** Robustly parse Python-based quantum circuits (Qiskit, Cirq, PennyLane) and convert them to OpenQASM 3.0 AST and source code.
- **Validation Layer:** Semantic analysis ensuring generated code adheres to the supported gate set and control flow limitations, with line-precise diagnostics.
- **Simulation Backends (QSim SDK):** Statevector, Density Matrix, and Stabilizer simulation via the independently deployable QSim SDK.
- **Interactive Web IDE:** A Monaco-based web editor for writing code, visualising circuits in real time, and managing projects.
- **Educational Features:** Gamified learning paths, hover-based quantum gate explanations, and community circuit sharing.
- **Cross-Framework Benchmarking:** Systematic performance comparison of Qiskit, Cirq, and PennyLane across 12 standard algorithms using QCanvas as the unified compilation platform.
- **SDK Packaging:** Both the QCanvas converter SDK and QSim simulator SDK packaged as installable Python packages (`pip install qcanvas-sdk`, `pip install qsim-sdk`).
- **Real-Time Feedback:** WebSocket-based streaming for long-running simulations with live progress updates.

1.3.2 Out of Scope

The following items were explicitly excluded to ensure feasibility within the project timeline:

- **Hardware Execution:** Direct integration with physical QPUs (e.g., IBM Quantum Experience) is not part of this release. The pluggable endpoint architecture supports this as a future extension without structural changes.
- **Pulse-Level Control:** Low-level control of microwave pulses (OpenPulse) is outside the scope, as it requires hardware-specific timing information not available in the simulator-first model.

- **Specialised QEC Protocols:** Quantum error-correction encoding and decoding layers beyond the Bit-Flip and Phase-Flip code examples are not implemented.
- **Physical Qubit Mapping:** Manual addressing of physical qubit layouts (e.g., $q[0]$ -> $q[0]$) is not supported, as the system targets logical-level simulation.

1.4 Modules

The system is architected into modular components to ensure scalability and maintainability across both the QCanvas platform and the QSim SDK.

1.4.1 Conversion Module

This module is the core translation engine of the system. It accepts Python source code written in any supported framework, uses an AST-based parser to extract circuit structure, and produces a valid OpenQASM 3.0 intermediate representation via the custom QASM3Builder pipeline.

1. AST-based parsing for Qiskit, Cirq, and PennyLane circuits with runtime fallback.
2. Intelligent gate mapping, parameter substitution, and qubit broadcast handling.
3. OpenQASM 3.0 code generation conforming to the supported subset.
4. Conversion statistics reporting: qubit count, gate depth, gate type histogram.

1.4.2 Simulation Module (QSim SDK)

The QSim module is the quantum execution kernel. Distributed as a standalone Python package, it serves as the computation backend for all circuit executions within QCanvas while remaining independently usable in research workflows.

1. Statevector, Density Matrix, and Stabilizer simulation backends.
2. Configurable shot counts (1–10,000) with deterministic seed support.
3. Intelligent result caching for repeated circuit executions.
4. Backend capability discovery: maximum supported qubits, available gate set.

1.4.3 Validation Module

This module acts as a gatekeeper before any circuit reaches the simulation engine. It performs semantic analysis on generated OpenQASM 3.0 output to prevent runtime failures caused by unsupported constructs.

1. OpenQASM 3.0 subset conformance checking.
2. Precise diagnostic reporting: line/column position, error code, remediation hint.
3. Detection and explanation of excluded features (timing, pulse-level control).

1.4.4 Web IDE Module

The Web IDE is the user-facing interface built on Next.js and Monaco Editor. It provides a complete quantum development environment accessible in any modern browser without installation.

1. Monaco-based code editor with custom syntax highlighting for Qiskit, Cirq, PennyLane, and OpenQASM 3.0.
2. Real-time D3.js circuit visualisation refreshed as the user types, with debounced updates to preserve responsiveness.
3. Interactive results panel with Chart.js histograms, statevector tables, and execution statistics.
4. Example library with over 40 categorised quantum algorithm implementations.

1.4.5 Educational Intelligence Module

This module transforms QCanvas from a development tool into a learning platform through contextual feedback, gamification, and community engagement.

1. “Quantum Explain It” hover system delivering gate matrices, Bloch-sphere explanations, and textbook context within the Monaco editor.
2. Event-driven gamification engine with 44 unique achievements, XP rewards, and a level progression system.
3. Community recipe-sharing platform for publishing, discovering, and remixing circuit snippets.

1.4.6 Benchmarking Module

The benchmarking module provides a systematic, reproducible pipeline for comparing quantum frameworks via unified compilation to OpenQASM 3.0 and execution on QSim.

1. 12 standard algorithm implementations, each written natively in Qiskit, Cirq, and PennyLane.
2. Compilation time, circuit depth, gate count, and multi-qubit ratio metrics per configuration.
3. Jensen–Shannon divergence analysis for pairwise cross-framework statistical equivalence.
4. QPack holistic scoring and Quantum Volume estimation across all tested frameworks.

1.5 Work Division

Team Member	Responsibilities
Umer Farooq	• FastAPI backend architecture, including API routing, JWT security middleware, and session management. • Hybrid CPU–QPU execution orchestration. • Backend job scheduling and result caching mechanisms. • Benchmarking suite design and analysis.
Hussan Waseem Syed	• Next.js Web IDE, including Monaco editor integration, D3.js circuit visualisation, and Chart.js results dashboards. • Educational Intelligence Module, including the gamification service, hover explanation system, and community recipe-sharing platform. • Frontend state management with Zustand. • Integration of the QSim simulation engine with the client-side pipeline.
Muhammad Irtaza Khan	• QCanvas Conversion Engine, including AST-based parsing, intelligent gate mapping, and the custom OpenQASM 3.0 generator. • Validation Module. • SDK packaging for the QCanvas converter. • Database design and persistence layer, including PostgreSQL schemas, SQLAlchemy ORM models, and Alembic migrations.

Table 1.2: Work Division

1.6 User Classes and Characteristics

User Class	Primary Goals	Key Needs
Beginners	Build visual intuition and learn quantum basics.	Templates, guided error messages, hover explanations, visual circuit feedback.
Researchers	Reproduce experiments and compare frameworks.	Batch processing, exact statevector access, cross-framework analysis, benchmark export.
Educators	Demonstrate concepts effectively to students.	Shareable examples, clean circuit visualisation, standard syntax compliance, example library.
SDK Developers	Programmatically integrate simulation into applications.	Pip-installable QSim SDK, REST API, structured response payloads, OpenAPI documentation.

Table 1.3: User Classes and Characteristics

1.7 Assumptions, Dependencies, and Constraints

1.7.1 Assumptions

Users are assumed to have basic familiarity with Python syntax. The Web IDE assumes a stable modern browser environment (Chrome/Firefox/Edge). SDK users are assumed to have Python 3.11+ installed.

1.7.2 Dependencies

Key external dependencies include Python 3.11+ for the backend, FastAPI for the API layer, Next.js 14 for the frontend, and the quantum libraries Cirq 1.0+, Qiskit 0.44+, and PennyLane 0.33+. Both the `qcanvas-sdk` and `qsim-sdk` packages are prepared as installable Python distributions.

1.7.3 Constraints

System performance is constrained by host memory for statevector simulations, which scale exponentially with qubit count (2^n complex amplitudes for n qubits). API response times are targeted at under 2 seconds for standard conversion tasks involving fewer than 100 gates. The stabilizer backend is limited to Clifford circuits by the Gottesman–Knill theorem.

CHAPTER 2

PROJECT REQUIREMENTS

This chapter describes the comprehensive functional and non-functional requirements for QCanvas and the QSim SDK, followed by use cases, external interface specifications, and system features that define the complete system behaviour across all four development iterations.

2.1 Functional Requirements

2.1.1 Conversion Module

The conversion module requirements define how source circuits are parsed, normalised, validated, and translated into OpenQASM 3.0. They also capture the statistics and diagnostics required for educational feedback and regression analysis.

- FR-C1: The system shall parse Cirq, Qiskit, and PennyLane circuits using AST-based parsing with runtime fallback.
- FR-C2: The system shall generate OpenQASM 3.0 code conforming to the supported subset per project scope [3].
- FR-C3: The system shall handle measurement and reset instructions correctly.
- FR-C4: The system shall expose conversion statistics including qubits, depth, and gate counts.
- FR-C5: The system shall validate generated OpenQASM 3.0 against the supported subset and provide detailed diagnostics [2].
- FR-C6: The system shall support gate mapping with parameter and broadcast handling.

2.1.2 Simulation Module (QSim SDK)

The simulation module requirements capture the execution behaviour of the QSim backends. The QSim SDK is packaged as an installable Python library and operates independently of the QCanvas Web IDE.

- FR-S1: The system shall support statevector, density matrix, and stabilizer simulation backends [5].
- FR-S2: The system shall allow configurable shots (1–10,000) and return counts and metadata.
- FR-S3: The system shall cache identical executions to improve performance.
- FR-S4: The system shall allow seed configuration for deterministic runs.
- FR-S5: The system shall provide backend capability discovery (max qubits, supported gates).
- FR-S6: The system shall return results with counts, execution time, and optional state snapshots.
- FR-S7: The QSim SDK shall be installable via `pip install qsdk` and importable as a standalone Python library.

2.1.3 Hybrid CPU–QPU Module

The hybrid module requirements describe how QCanvas separates classical orchestration from quantum execution, keeping the frontend responsive while heavy computation is delegated to the QSim backend.

- FR-H1: The system shall separate classical orchestration (CPU) from quantum circuit execution (QPU/Simulator) [6].
- FR-H2: The system shall implement data flow: Source $\rightarrow$ QASM $\rightarrow$ Execution $\rightarrow$ Results.
- FR-H3: The system shall provide WebSocket progress updates for long-running operations.
- FR-H4: The system shall enforce simulation-first execution with optional pluggable endpoints.
- FR-H5: The system shall provide resource-aware execution with limits and timeouts.

2.1.4 Web IDE Module

- FR-W1: The system shall provide code editing with syntax highlighting and diagnostics via Monaco Editor.
- FR-W2: The system shall display real-time D3.js circuit visualisation and Chart.js results panels.
- FR-W3: The system shall support multiple example projects and templates.
- FR-W4: The system shall provide keyboard shortcuts for frequent actions (e.g., Ctrl+Shift+R to run).
- FR-W5: The system shall support multi-file projects with workspace management.

2.1.5 Educational Intelligence Module

- FR-E1: The system shall provide hover-based contextual explanations for quantum gates, algorithms, and concepts within the Monaco editor.
- FR-E2: The system shall implement a gamification engine with XP rewards, level progression, and at least 44 unique achievements.
- FR-E3: The system shall allow users to publish, discover, and remix community circuit snippets with metadata (framework, difficulty, tags).
- FR-E4: The system shall track user learning progress with activity history and achievement dashboards.

2.1.6 API and Realtime

- FR-A1: The system shall expose REST endpoints for conversion and validation.
- FR-A2: The system shall expose WebSocket endpoints for progress streaming.
- FR-A3: The system shall provide structured error payloads with codes and hints.
- FR-A4: The system shall provide discovery endpoints for frameworks and backends.
- FR-A5: The system shall support batch conversion and simulation operations.

2.1.7 Validation

- **FR-V1:** The system shall validate OpenQASM 3.0 output against the allowed subset.
- **FR-V2:** The system shall suggest nearest supported constructs for incompatibilities.
- **FR-V3:** The system shall emit precise diagnostics with line/column, code, and remediation.
- **FR-V4:** The system shall check for excluded features (timing, pulse-level) with clear guidance.

2.2 Non-Functional Requirements

2.2.1 Reliability

- **NFR-R1: Service Availability**
The system containers (Backend, Database, Redis) shall report a “healthy” status (200 OK) on their health-check endpoints 99.9% of the time during active operation.
- **NFR-R2: Audit Trail**
The system shall record an audit log entry for 100% of API requests resulting in 4xx or 5xx status codes.

2.2.2 Usability

- **NFR-U1: Keyboard Efficiency**
The system shall execute the current quantum circuit when the user triggers Ctrl+Shift+R (or Cmd+Shift+R).
- **NFR-U2: Visual Accessibility**
The user interface shall utilise the Monaco Editor’s high-contrast vs-dark theme standards to ensure text legibility.
- **NFR-U3: Hover Response**
The “Quantum Explain It” hover provider shall respond within 200ms of cursor placement on a recognised symbol.

2.2.3 Performance

- **NFR-P1: Conversion Latency**

The API shall process circuit conversion requests (AST parsing to QASM generation) in under 1500ms for circuits with <100 gates.

- **NFR-P2: Simulation Throughput**

The QSim backend shall complete execution of standard algorithms (e.g., Grover's Search, $n = 2$) in under 5 seconds.

- **NFR-P3: Visualisation Render Time**

The D3.js circuit visualisation shall render updates in under 200ms for circuits up to 50 gates.

2.2.4 Security

- **NFR-S1: API Rate Limiting**

The API shall enforce a rate limit of **20 requests per minute** per client IP address for conversion endpoints.

- **NFR-S2: Data Integrity Protection**

The system shall sanitise 100% of user inputs via ORM parameterisation to prevent SQL injection vulnerabilities.

- **NFR-S3: Security Properties**

Confidentiality: Sensitive API keys shall be encrypted at rest using AES-256, and Role-Based Access Control (RBAC) shall ensure each role can only access permitted resources. *Integrity:* User passwords shall be hashed using Bcrypt with per-user salts, and all state transitions shall be validated server-side. *Availability:* The system shall use rate limiting, health checks, and resource controls to reduce service disruption during abnormal traffic conditions.

2.3 External Interface Requirements

2.3.1 User Interfaces

2.3.1.1 Web IDE Interface

- Monaco-based code editor with syntax highlighting for Qiskit, Cirq, PennyLane, and OpenQASM 3.0.
- Multi-panel layout: Editor, Circuit Visualisation, Results, Console, Statistics.
- File tree navigation with create, rename, and delete operations.
- Theme switcher (dark/light modes) with responsive design for desktop and tablet.

2.3.1.2 Results and Visualisation

- Real-time D3.js 2D and 3D circuit diagram updated as the user types.
- Tabbed results display: Console, Output, Histogram, OpenQASM, Errors, Stats.
- Interactive measurement histograms using Chart.js.
- Export functionality for results (JSON, CSV) and QASM files.

2.3.2 Software Interfaces

2.3.2.1 Quantum Framework APIs

- Qiskit API (version 0.44+): Circuit construction and manipulation [8].
- Cirq API (version 1.0+): Circuit operations and gate definitions [4].
- PennyLane API (version 0.33+): Quantum node processing [1].

2.3.3 Communication Interfaces

2.3.3.1 REST API

- Protocol: HTTPS in production, HTTP in development.
- Format: JSON request/response bodies.

- Authentication: JWT tokens with RBAC.
- Base URL: /api/v1/

2.3.3.2 WebSocket Interface

- Protocol: WSS in production, WS in development.
- Endpoint: /ws
- Use cases: Progress updates, real-time simulation notifications.

Table 2.1 summarises the principal runtime events exchanged between the user interface and the backend services that implement QCanvas.

Table 2.1: QCanvas Event-Response Matrix

ID	Event / Trigger	Source	Handling Subsystem	System Response	Destination
E-1	Submit login credentials	User Browser	FastAPI Auth Service	Verify credentials; generate JWT authentication token	Next.js Client
E-2	Save active quantum code	User Browser	File/Project Service	Persist current workspace file state	PostgreSQL
E-3	Initiate circuit conversion	User Browser	QCanvas SDK (Converters)	Parse AST; map gates; generate target syntax (OpenQASM)	Monaco Editor
E-4	Execute simulation job	User Browser	QSim SDK (Simulator)	Dispatch QASM to selected backend; cache output metrics	Chart.js Visuals
E-5	Simulation progress update	Simulation Engine	WebSocket Service	Stream job state and elapsed time indicators	Progress Bar
E-6	Request hover explanation	User Browser	Explain-It Service	Retrieve contextual gate explanation and educational metadata	Monaco Hover Panel
E-7	Unlock milestone/goal	Core Engine	Gamification Service	Grant user XP; recalculate user levels	Achievement Panel

2.4 Use Cases

The following use cases describe the core interactions between users and the QCanvas system.

2.4.1 Functional Requirements Traceability

Table 2.2: Traceability between FRs and Use Cases

FR Group	Primary Related Use-Cases
Conversion (FR-C1–FR-C6)	UC-02, UC-03, UC-06, UC-07
Simulation (FR-S1–FR-S7)	UC-04, UC-05, UC-09, UC-10
Hybrid CPU–QPU (FR-H1–FR-H5)	UC-04, UC-10
Web IDE (FR-W1–FR-W5)	UC-01, UC-02, UC-05, UC-08
Educational (FR-E1–FR-E4)	UC-11, UC-12, UC-13
API and Realtime (FR-A1–FR-A5)	UC-03, UC-04, UC-09
Validation (FR-V1–FR-V4)	UC-03 (error paths), UC-06

2.4.2 UC-01: Launch Web IDE

- **Primary Actor:** User
- **Precondition:** Browser has network access to the QCanvas URL.
- **Main Flow:**
 1. User navigates to the application URL.
 2. System loads the Monaco editor, sidebar, and results panel.
 3. System fetches the default “Hello World” circuit.
 4. System displays the “Ready” status indicator.
- **Postcondition:** IDE is initialised and waiting for input.

2.4.3 UC-02: Write Circuit Code

- **Primary Actor:** User
- **Precondition:** UC-01 complete; editor is active.

- **Main Flow:**
 1. User selects a target framework (e.g., Qiskit) from the dropdown.
 2. User types quantum circuit code into the editor.
 3. System performs real-time syntax checking and updates D3.js circuit diagram.
- **Postcondition:** Valid or invalid source code exists in the editor buffer.

2.4.4 UC-03: Convert to OpenQASM

- **Primary Actor:** User
- **Precondition:** Source code is present in the editor.
- **Main Flow:**
 1. User clicks the “Compile” button.
 2. System sends source code to the Conversion API.
 3. API parses the code and generates OpenQASM 3.0 intermediate representation.
 4. System returns the QASM code and displays it in the output panel.
- **Alternate Flows:**
 - 3a *Syntax Error:* API returns line number; system highlights the error line in red.
 - 3b *Unsupported Gate:* System returns a gate-not-supported error with a remediation hint.
- **Postcondition:** OpenQASM 3.0 code is available for simulation.

2.4.5 UC-04: Execute Circuit

- **Primary Actor:** User
- **Precondition:** Valid OpenQASM 3.0 code exists in the editor or output panel.
- **Main Flow:**
 1. User clicks “Run” and configures parameters (Shots=1024, Backend=Statevector).
 2. System submits job to the QSim backend.
 3. QSim executes the circuit and returns measurement results.

2. Project Requirements

4. System pushes results to the frontend via WebSocket.

- **Alternate Flows:**

2a *Timeout*: Simulation exceeds 30s. System displays a timeout message with retry option.

- **Postcondition**: Measurement counts and state data are available in the client.

2.4.6 UC-05: Visualise Results

- **Primary Actor**: User

- **Precondition**: UC-04 completed successfully.

- **Main Flow**:

1. System receives JSON result payload.
2. System renders a bar chart using Chart.js for measurement counts.
3. System displays execution metadata (time, backend version, qubit count).

- **Alternate Flows**:

2a *Empty Result*: If 0 shots were requested, system displays statevector only without histogram.

- **Postcondition**: Results are visible to the user.

2.4.7 UC-06: Handle Validation Errors

- **Primary Actor**: User

- **Precondition**: User has triggered Conversion or Execution with invalid or unsupported input.

- **Main Flow**:

1. System detects a semantic error (e.g., unsupported gate, timing construct).
2. System squiggles the error line in red in the editor.
3. System displays a “Problems” pane with error codes (e.g., ERR-020) and remediation suggestions.

- **Alternate Flows**:

3a *Manual Correction*: User edits the code to resolve the issue. System re-validates and clears the error indicator.

- **Postcondition:** User is aware of the exact location and nature of the error.

2.4.8 UC-07: Export OpenQASM

- **Primary Actor:** User
- **Precondition:** UC-03 completed successfully; QASM code is available.
- **Main Flow:**
 1. User clicks the “Download QASM” icon in the output panel.
 2. System generates a .qasm file including a metadata header.
 3. System triggers the browser file download dialog.
- **Alternate Flows:**

3a *File Size Limit*: If code exceeds 1MB, system compresses the content before download.
- **Postcondition:** The OpenQASM source file is saved to the user’s local file system.

2.4.9 UC-08: Load Example Circuit

- **Primary Actor:** User
- **Precondition:** IDE is active.
- **Main Flow:**
 1. User opens the “Examples” drawer.
 2. User clicks on an algorithm (e.g., “Grover’s Algorithm”).
 3. System replaces current editor content with the selected example’s source code.
 4. System displays a modal with a brief explanation of the algorithm.
- **Alternate Flows:**

3a *Dirty Buffer*: If current code is unsaved, system prompts “Discard changes?”.
- **Postcondition:** Example code is loaded and ready to run.

2.4.10 UC-09: Query Backend Capabilities

- **Primary Actor:** Developer
- **Precondition:** Authenticated API access is available.
- **Main Flow:**
 1. Developer requests GET /api/v1/backends.
 2. System returns list of available backends (e.g., “statevector”, “stabilizer”).
 3. System details max_qubits (e.g., 20) and supported_gates for each.
- **Alternate Flows:**
 - 2a *Database Down:* System returns a cached fallback capabilities list.
- **Postcondition:** Developer client is configured to respect backend limits.

2.4.11 UC-10: Hybrid CPU–QPU Execution

- **Primary Actor:** User/System
- **Precondition:** Circuit has been submitted for execution.
- **Main Flow:**
 1. **Orchestration (CPU):** Validates QASM and allocates a Job ID.
 2. **Orchestration (CPU):** Dispatches payload to the QSim Service.
 3. **Execution (QSim):** Initialises the quantum statevector.
 4. **Execution (QSim):** Applies gate operations sequentially.
 5. **Orchestration (CPU):** Receives final counts and saves to database.
- **Alternate Flows:**
 - 3a *Memory OOM:* QSim detects >20 qubits. Returns “Resource Limit Exceeded”; CPU relays error to user.
- **Postcondition:** Job is marked as “Completed” or “Failed” in the database.

2.4.12 UC-11: Hover Explanation (Quantum Explain It)

- **Primary Actor:** User (learner)
- **Precondition:** IDE is active with quantum source code in the editor.
- **Main Flow:**
 1. User hovers cursor over a gate or algorithm keyword (e.g., H, CNOT, VQE).
 2. System looks up the explanation for the recognised symbol.
 3. System renders a markdown popover with the gate matrix, Bloch-sphere implications, and a textbook explanation.
- **Postcondition:** User receives contextual quantum education without leaving the editor.

2.4.13 UC-12: Earn Achievement

- **Primary Actor:** User
- **Precondition:** User is authenticated and performing qualifying activities.
- **Main Flow:**
 1. User completes a qualifying activity (e.g., 50 simulation runs, sharing a recipe).
 2. Gamification service evaluates the activity against 44 achievement criteria.
 3. System awards XP, updates the user's level, and displays an achievement notification.
- **Postcondition:** Achievement is recorded and displayed on the user's profile dashboard.

2.4.14 UC-13: Share Circuit Recipe

- **Primary Actor:** User
- **Precondition:** User is authenticated; circuit code is present in the editor.
- **Main Flow:**
 1. User opens the Share modal and annotates the snippet with title, difficulty, framework, and tags.

2. Project Requirements

2. System publishes the recipe to the community library.
 3. Other users can discover, fork, and run the shared circuit.
- **Postcondition:** Recipe is publicly visible in the community library.

2.5 Use Cases Diagram

This diagram outlines the primary actors and their interactions with QCanvas across core flows: conversion, simulation, educational features, and community sharing.

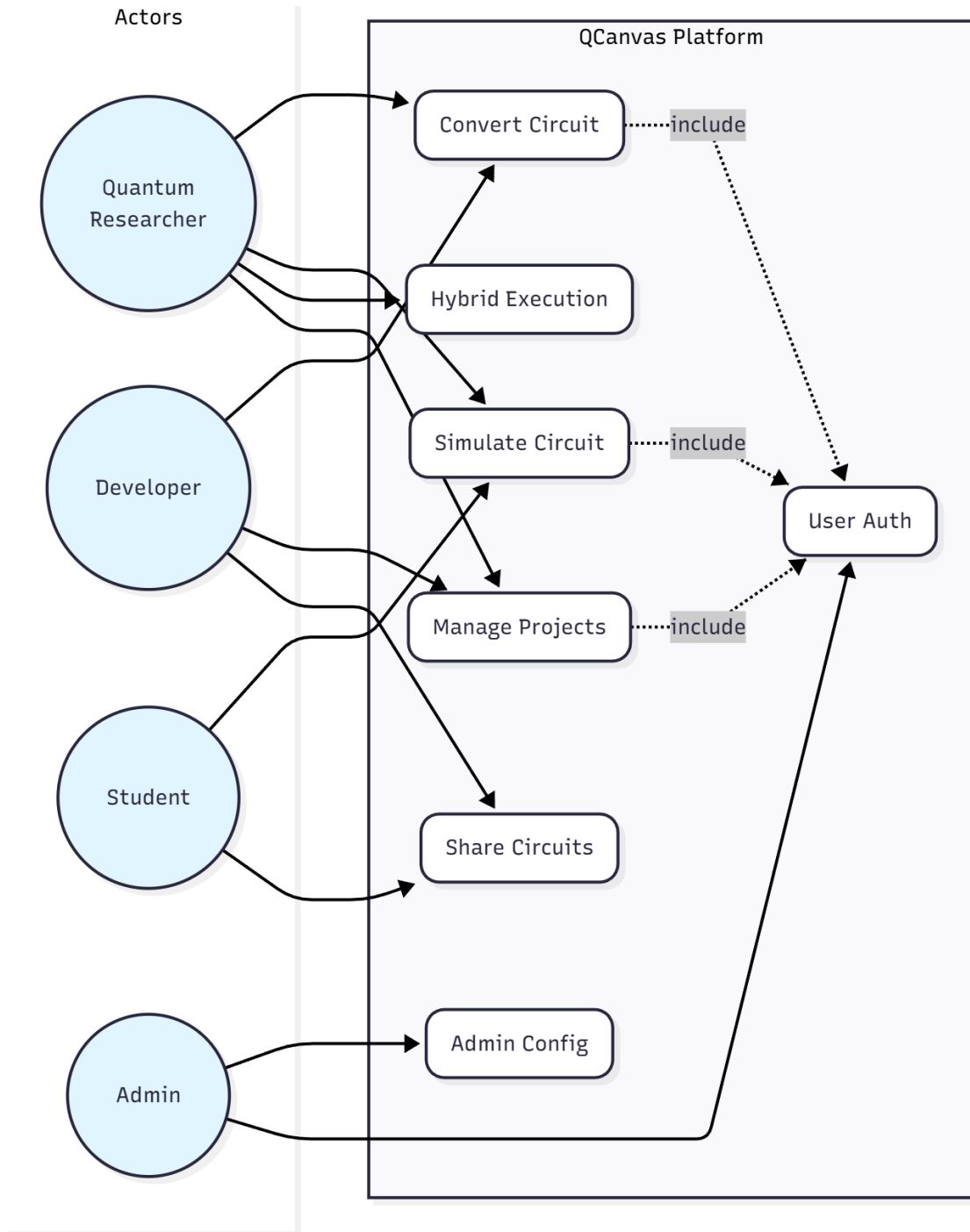


Figure 2.1: Use Case Diagram

2. Project Requirements

2.6 Domain Model Diagram

This diagram shows the core entities (Circuit, ConversionRequest, SimulationJob, SimulationResult, User, Project, Achievement, Recipe) and key relationships that support conversion, simulation, and educational features.

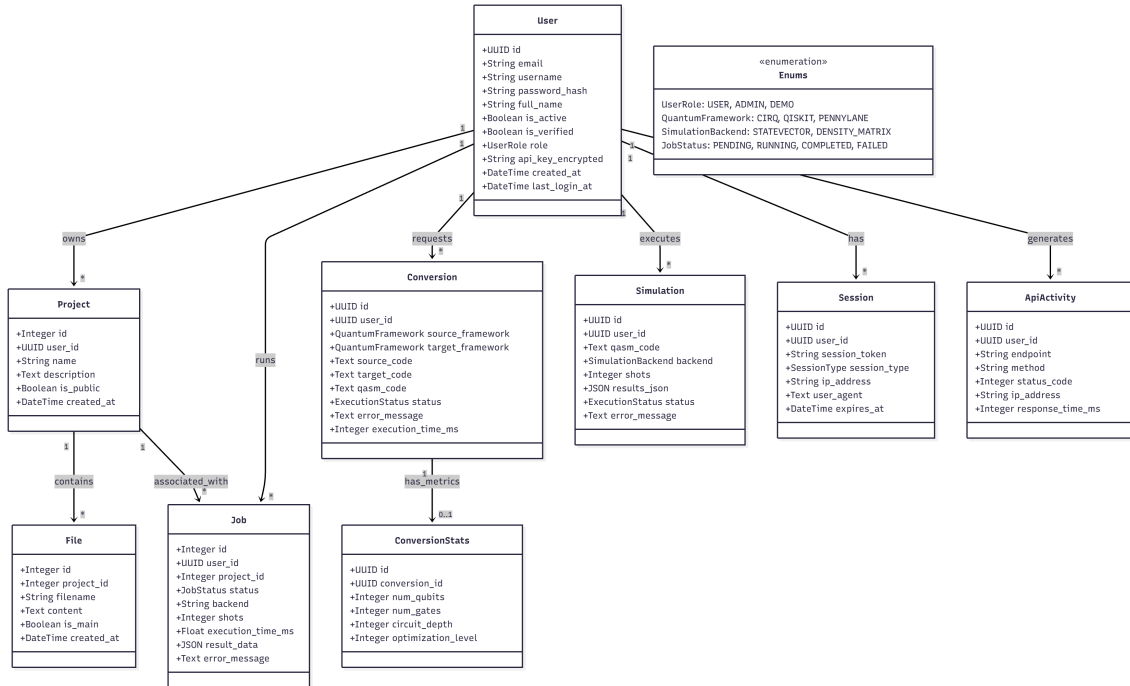


Figure 2.2: Domain Model

CHAPTER 3

SYSTEM OVERVIEW

By the conclusion of Iteration 4, QCanvas has evolved into a multi-layer platform in which the QCanvas Web IDE, the QCanvas conversion SDK, and the QSim simulation SDK each operate as distinct, independently deployable components connected by a FastAPI API layer. This chapter describes the final architecture, design models, and data design of the completed system.

3.1 Architectural Design

QCanvas employs a layered hybrid architecture in which the Next.js frontend handles the user interface and lightweight operations, the FastAPI backend manages compilation and validation, and the QSim SDK executes circuits on simulation backends. Both the `qcanvas-sdk` and `qsim-sdk` are packaged as installable Python tools, making their capabilities accessible beyond the Web IDE.

Figure 3.1 presents a simplified box-and-line view of the major QCanvas subsystems before the more detailed architecture, component, and deployment models discussed later in this chapter.

3. System Overview

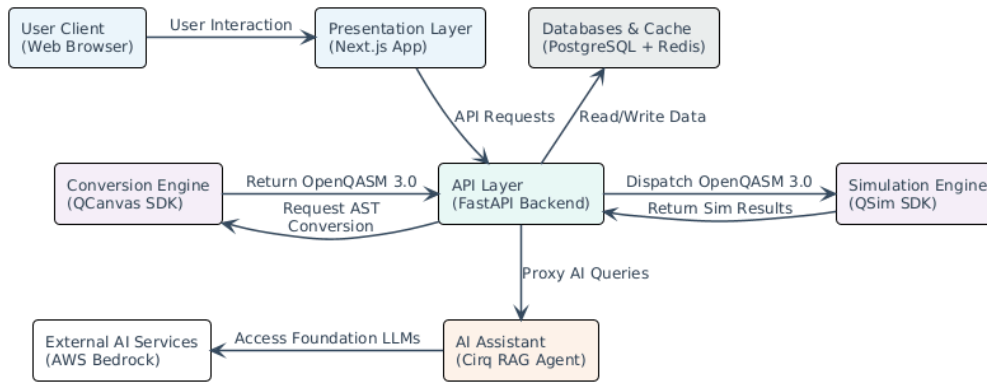


Figure 3.1: Simplified Box-and-Line Architecture of QCanvas

3.1.1 Layered Responsibilities

- **Presentation Layer (Next.js):** Monaco editor UI, D3.js circuit visualisation, Chart.js results dashboards, gamification achievement panels, community recipe library, and client-side state management via Zustand.
- **API Layer (FastAPI):** REST endpoints for conversion, validation, and simulation; WebSocket progress streams; structured error handling; JWT authentication; and rate limiting.
- **Conversion Layer (QCanvas SDK):** AST parsing, intelligent gate mapping, OpenQASM 3.0 generation via QASM3Builder, validation, conversion analytics, and structured logging.
- **Execution Layer (QSim SDK):** Simulation backends (statevector, density matrix, stabilizer), job scheduling, result aggregation, caching, and performance metrics. Independently installable via `pip install qsim-sdk` [10].
- **Persistence Layer:** PostgreSQL for user accounts, projects, files, conversion history, and achievements; Redis for caching and session management.
- **Observability Layer:** Health checks, Prometheus metrics, execution time histograms, and structured logging via Python’s logging framework.

3.1.2 Technology Stack

3.1.2.1 Frontend Technologies

- Next.js 14 with App Router for server-side rendering and routing.
- React 18 with TypeScript for type-safe UI component development.
- Tailwind CSS for styling with a custom quantum-themed design system.
- Monaco Editor for in-browser code editing with custom syntax highlighting and hover providers.
- D3.js for 2D circuit visualisation; @react-three/fiber for 3D rendering.
- Zustand for lightweight client-side state management.
- Chart.js for measurement histogram visualisation.

3.1.2.2 Backend Technologies

- Python 3.11+ for backend logic and quantum library integration.
- FastAPI for the high-performance async API framework.
- Uvicorn ASGI server for production deployment.
- Pydantic for data validation and serialisation.
- SQLAlchemy ORM with PostgreSQL.
- Redis for caching and session management.
- Alembic for schema migrations.

3.1.2.3 Quantum Computing Stack

- Cirq 1.0+ for Google quantum framework support [4].
- Qiskit 0.44+ for IBM quantum framework support [8].
- PennyLane 0.33+ for Xanadu quantum framework support [1].
- Custom OpenQASM 3.0 generator (QASM3Builder) [3].
- QSim SDK: Custom statevector, density matrix, and stabilizer backends [5, 10].

3.1.3 Hybrid CPU–QPU Execution Model

3.1.3.1 Orchestration Boundary

- **CPU Responsibilities:** Circuit construction, parameter preparation, AST parsing, OpenQASM generation, validation, pre/post-processing, and result aggregation.
- **QSim Responsibilities:** Gate-level circuit execution, statevector/density matrix evolution, measurement sampling, and state snapshot generation.

3.1.3.2 Data Flow

1. User writes circuit in source framework (Qiskit/Cirq/PennyLane) in the Monaco editor.
2. Frontend sends code to FastAPI backend via REST API.
3. Backend parses code using AST or runtime execution (detection by import inspection).
4. QCanvas SDK generates OpenQASM 3.0 intermediate representation via QASM3Builder.
5. Validation layer checks conformance to the supported gate subset and control flow.
6. QSim SDK executes circuit on selected backend (statevector/density/stabilizer).
7. Results are aggregated and returned to frontend via REST response and WebSocket push.
8. Frontend visualises results: histograms, circuit diagrams, execution statistics.

3.2 Design Models

3.2.1 Activity Diagrams

3.2.1.1 Activity Diagram: Circuit Conversion and Execution Pipeline

This diagram presents the end-to-end flow from code input in the Monaco editor, through FastAPI conversion and validation services, into QSim execution and results visualisation.

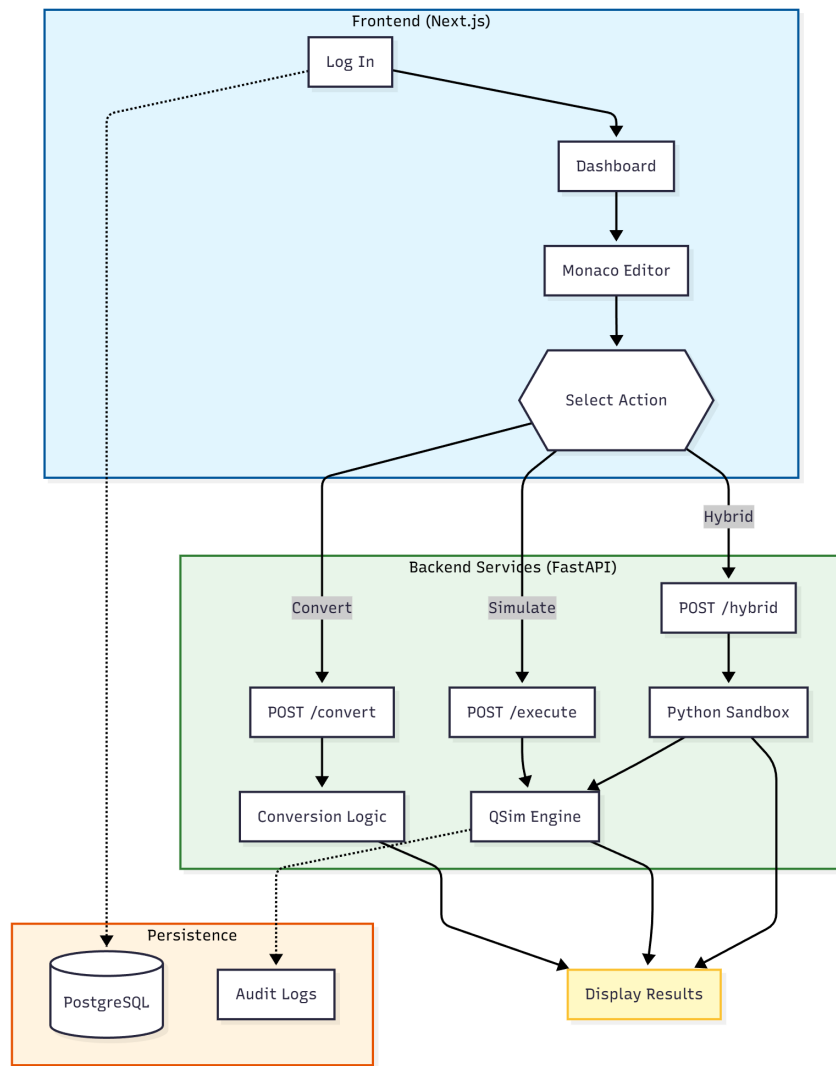


Figure 3.2: Activity Diagram — Circuit Conversion and Execution Pipeline

3.2.1.2 Activity Diagram: User Registration and Session Issuance

This diagram captures the user registration flow: a new user submits credentials, FastAPI validates and hashes the password using Bcrypt, a PostgreSQL record is created, and a JWT session token is issued.

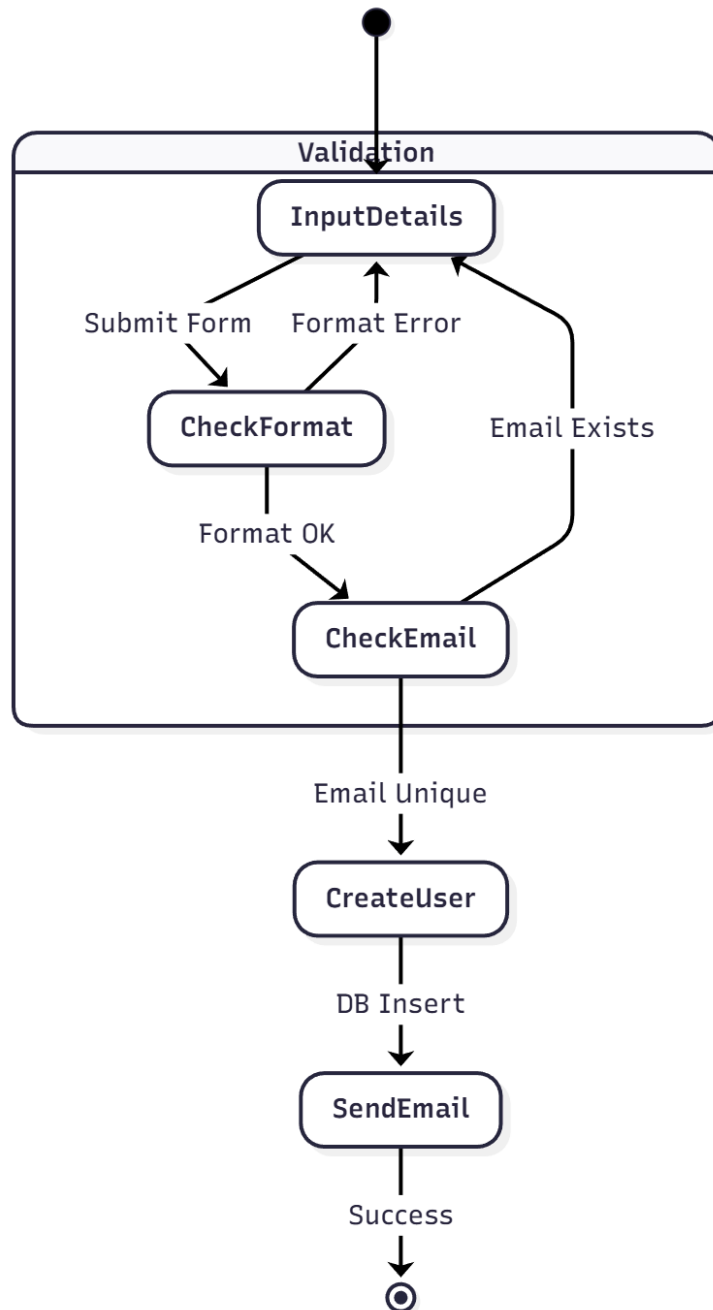


Figure 3.3: Activity Diagram — User Registration and Session Issuance

3.2.2 Sequence Diagrams

3.2.2.1 Sequence Diagram: Unified Execution Workflow

This diagram shows the complete interaction across system components: user browser, Next.js frontend, FastAPI routes, ConversionService, SimulationService, and QSim back-ends.

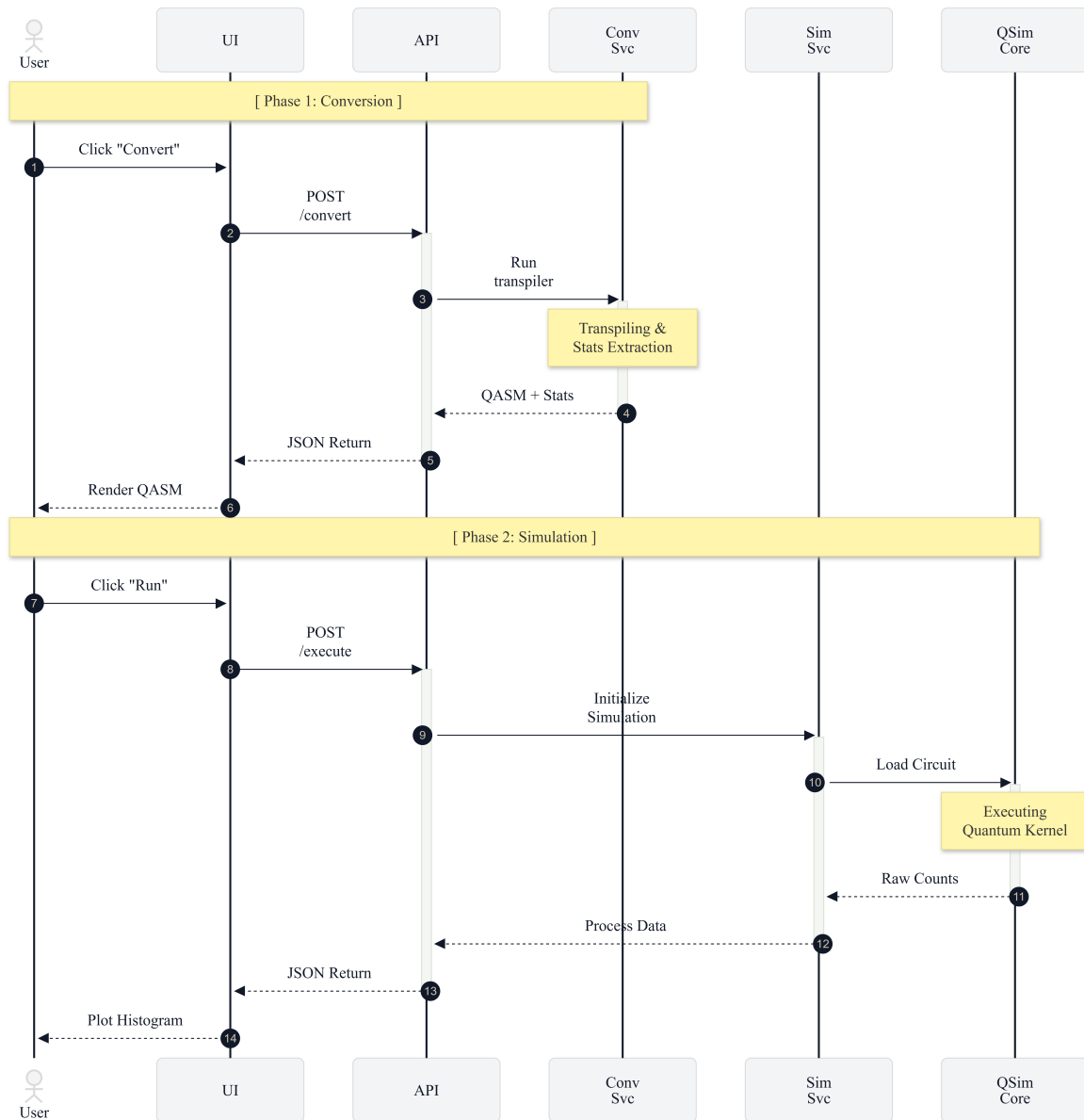


Figure 3.4: Sequence Diagram — Unified Execution Workflow

3.2.2.2 Sequence Diagram: JWT Authentication Handshake

This diagram shows the JWT-based authentication handshake between client, FastAPI, and PostgreSQL.

3. System Overview

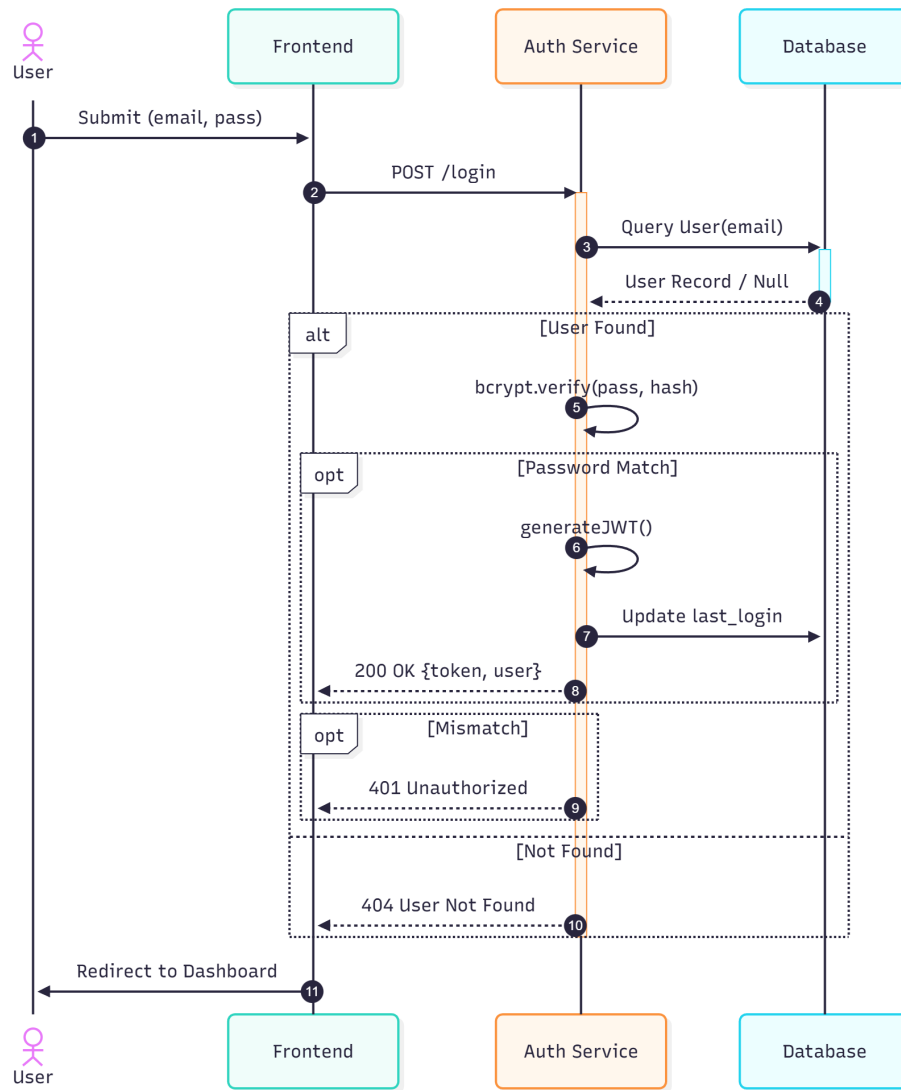


Figure 3.5: Sequence Diagram — JWT Authentication Handshake

3.2.2.3 System Sequence Diagrams (SSD)

SSD: Authentication Handler — illustrates the system boundary interactions for user login, token issuance, and session validation.

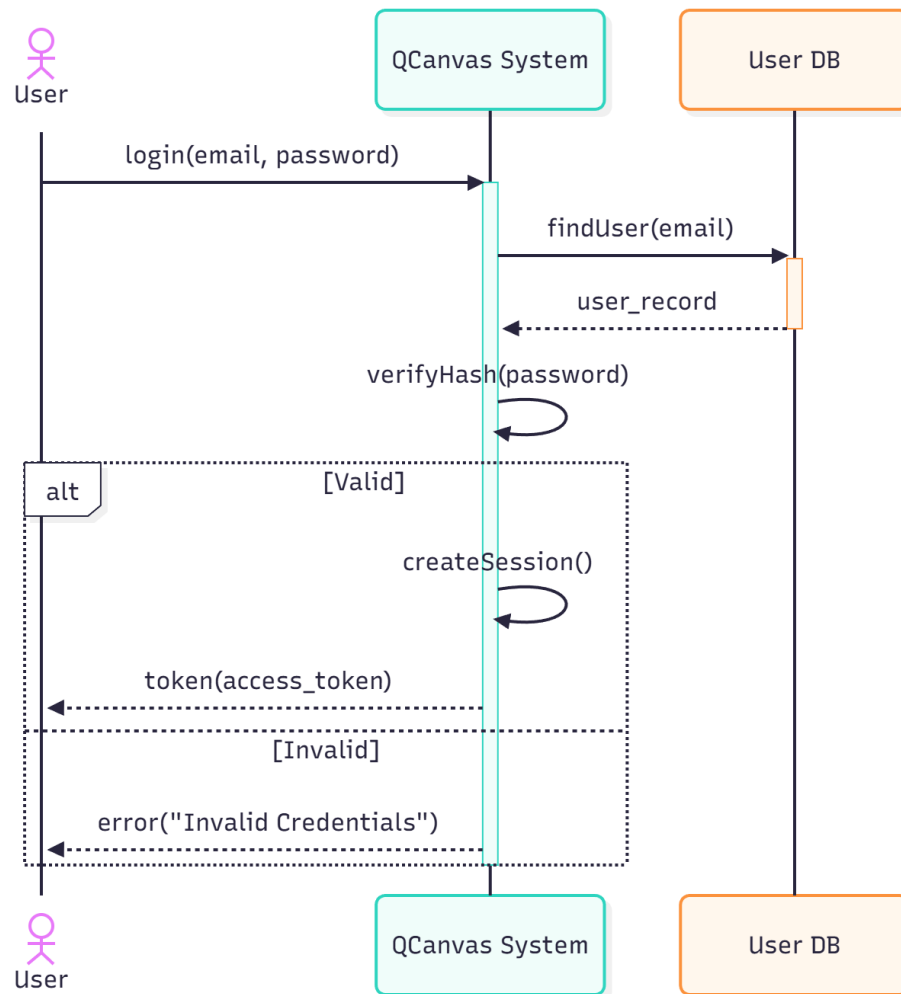


Figure 3.6: SSD — Authentication Handler

3. System Overview

SSD: Circuit OpenQASM Compilation — shows the external interactions for the circuit compilation flow from the user's perspective.

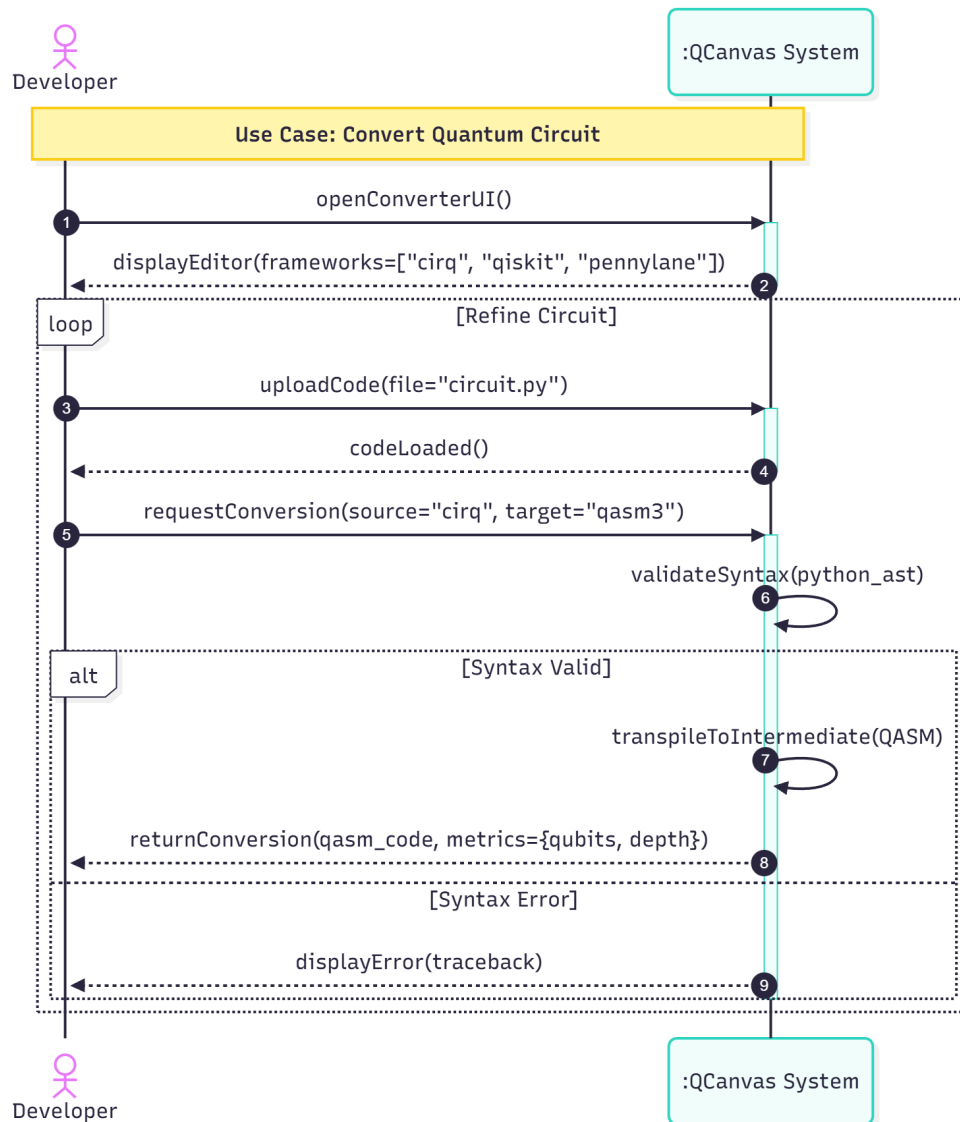


Figure 3.7: SSD — Circuit OpenQASM Compilation

SSD: QSim Backend Simulation Execution — depicts the execution handoff from the user action through the API to QSim and back.

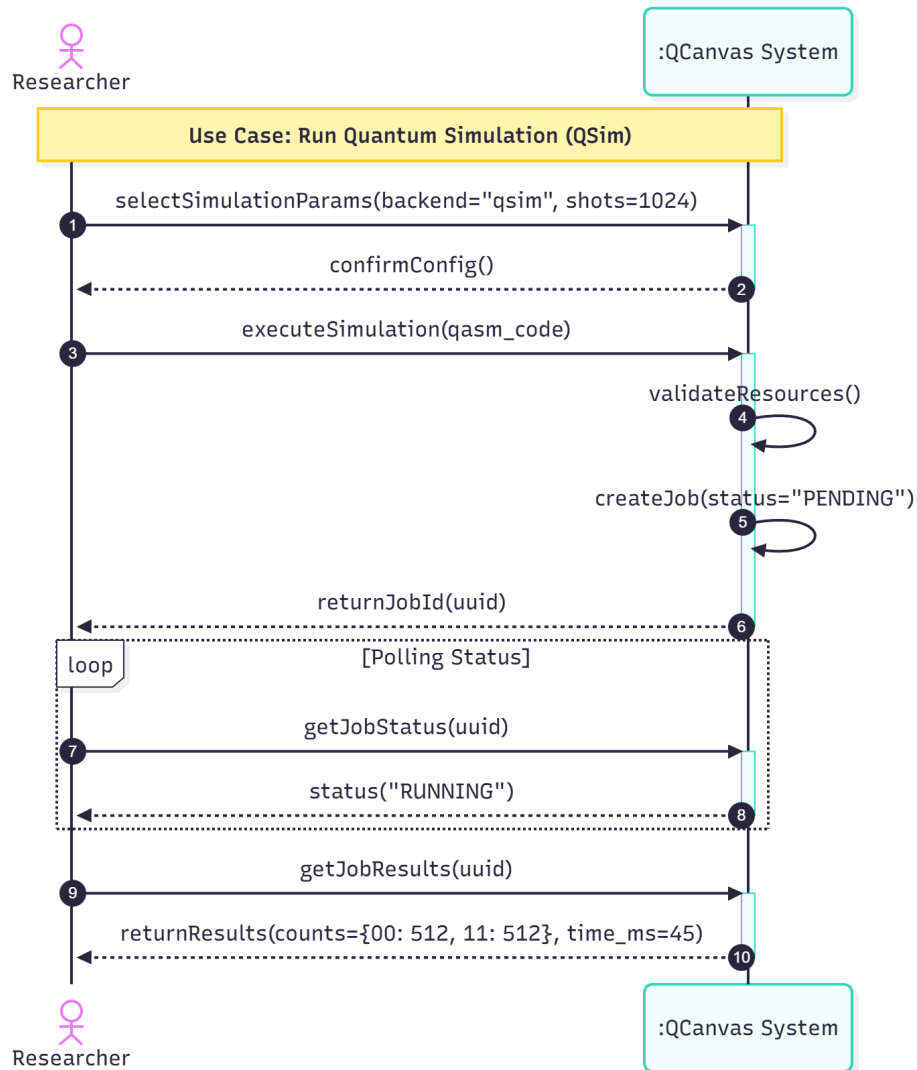


Figure 3.8: SSD — QSim Backend Simulation Execution

3. System Overview

3.2.3 Architecture Diagram

This diagram provides a high-level view of the deployed QCanvas stack, showing the Next.js IDE, FastAPI API layer, quantum_converters (QCanvas SDK) and quantum_simulator (QSim SDK) packages, PostgreSQL/Redis persistence, and Web-Socket/REST communication channels.

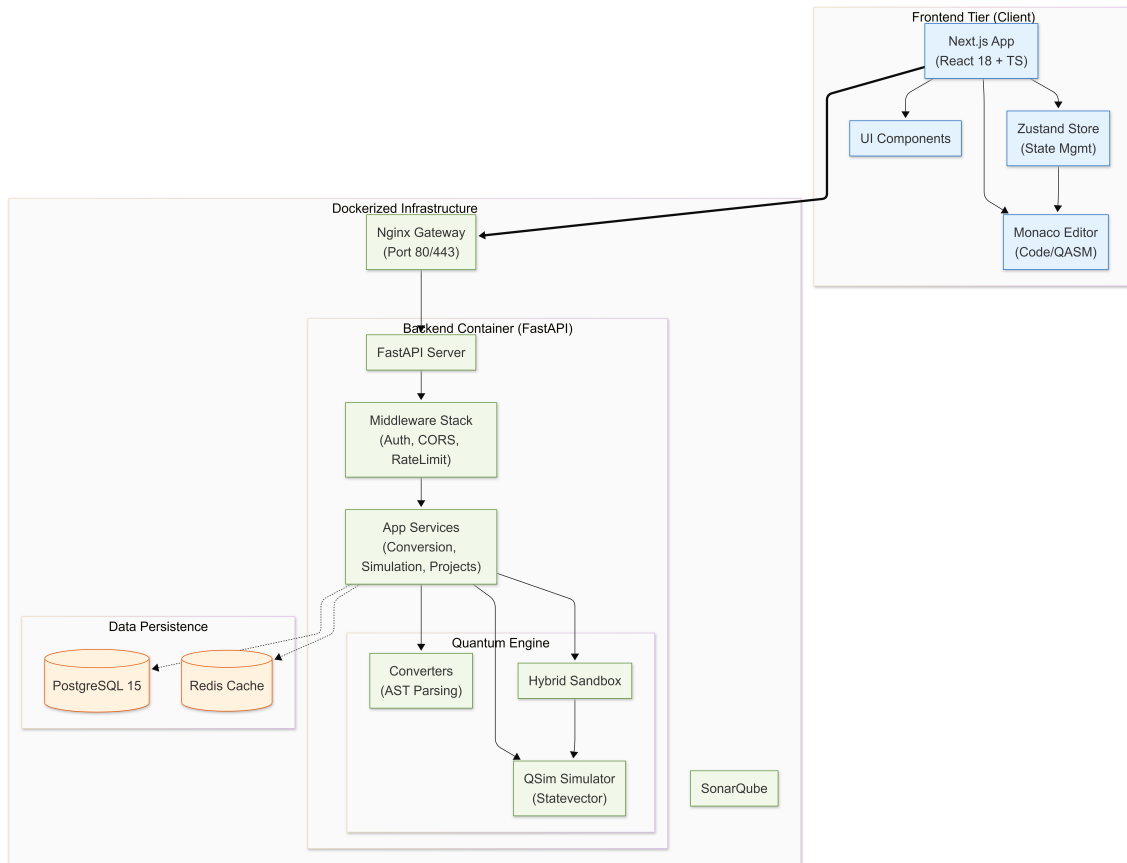


Figure 3.9: System Architecture Diagram

3.2.4 Component Diagram

This diagram depicts concrete module boundaries: editor and results components in the frontend, API routes and services in the backend, conversion and validation modules in the QCanvas SDK, QSim simulation backends, and shared utilities.

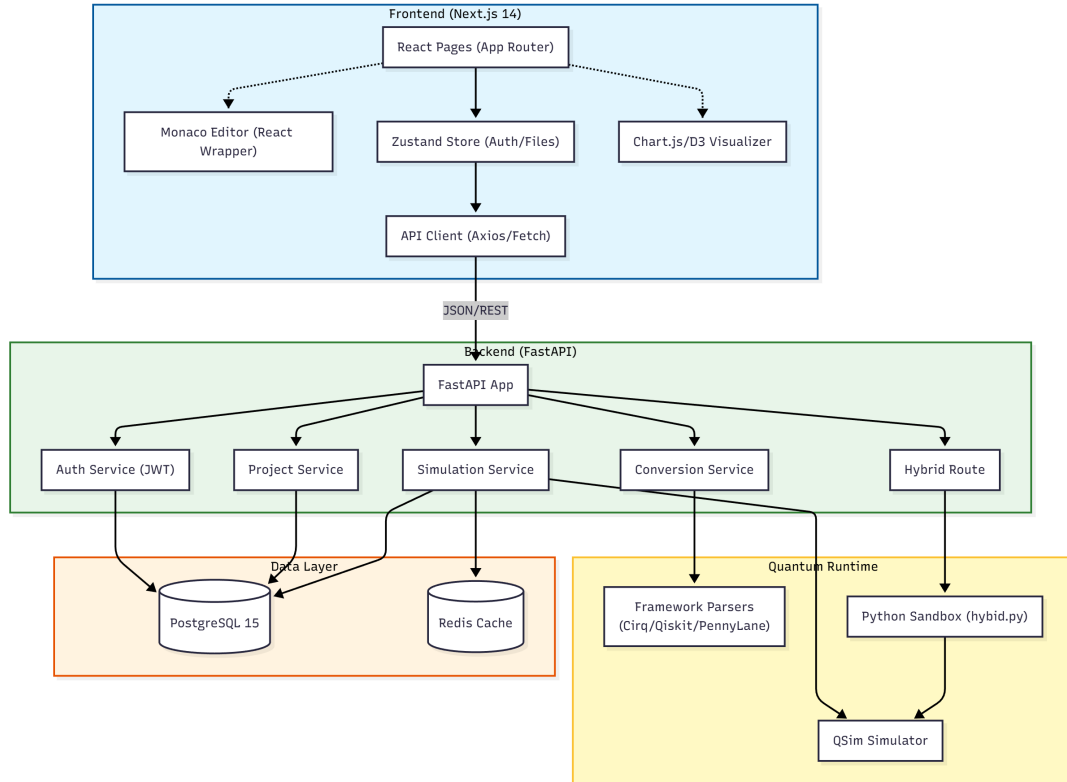


Figure 3.10: Component Diagram

3.2.5 Package Diagram

This diagram outlines the package organisation across the frontend (Next.js app, components, lib), backend (app/api, core, services, models), quantum_converters (parsers, converters, optimizers), and quantum_simulator (QSim) libraries.

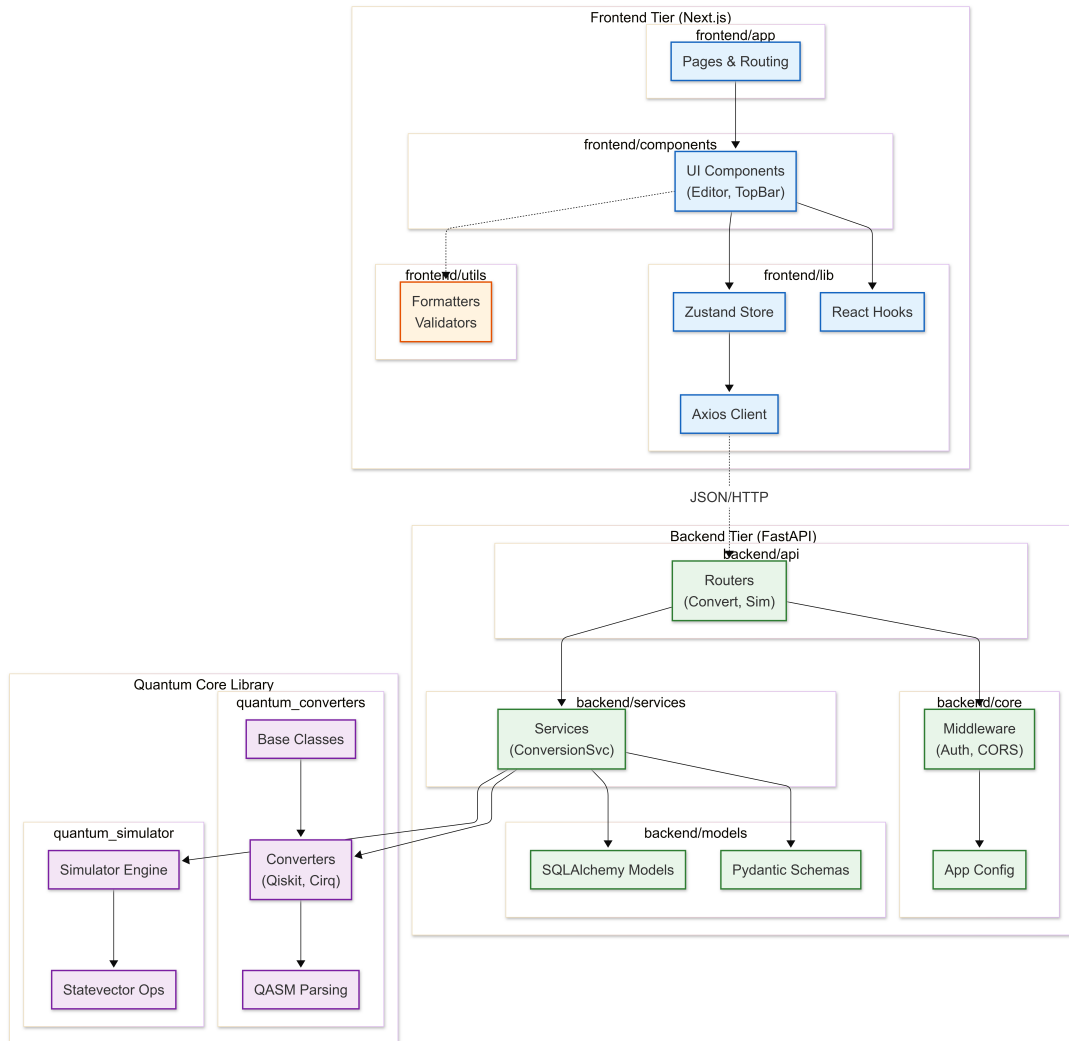


Figure 3.11: Package Diagram

3.2.6 Deployment Diagram

This diagram shows the production deployment topology: Nginx as the HTTP reverse proxy, the FastAPI backend service, Next.js frontend, PostgreSQL and Redis containers, and containerisation via Docker Compose.

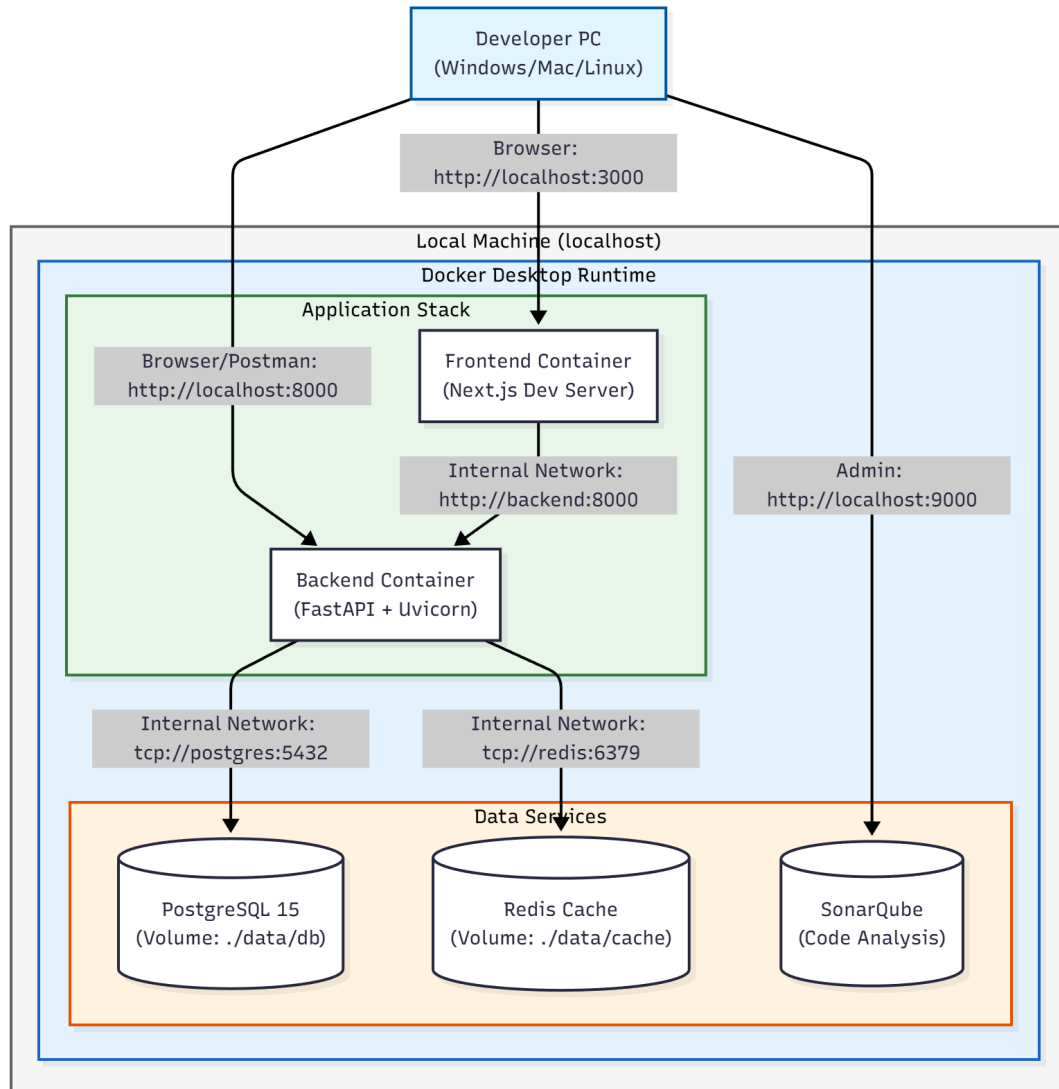


Figure 3.12: Deployment Diagram

3.2.7 State Diagrams

3.2.7.1 State Diagram: Conversion Process

This diagram shows the state machine for a conversion request: Idle → Parsing → Mapping → Generating → Validating → Success/Error.

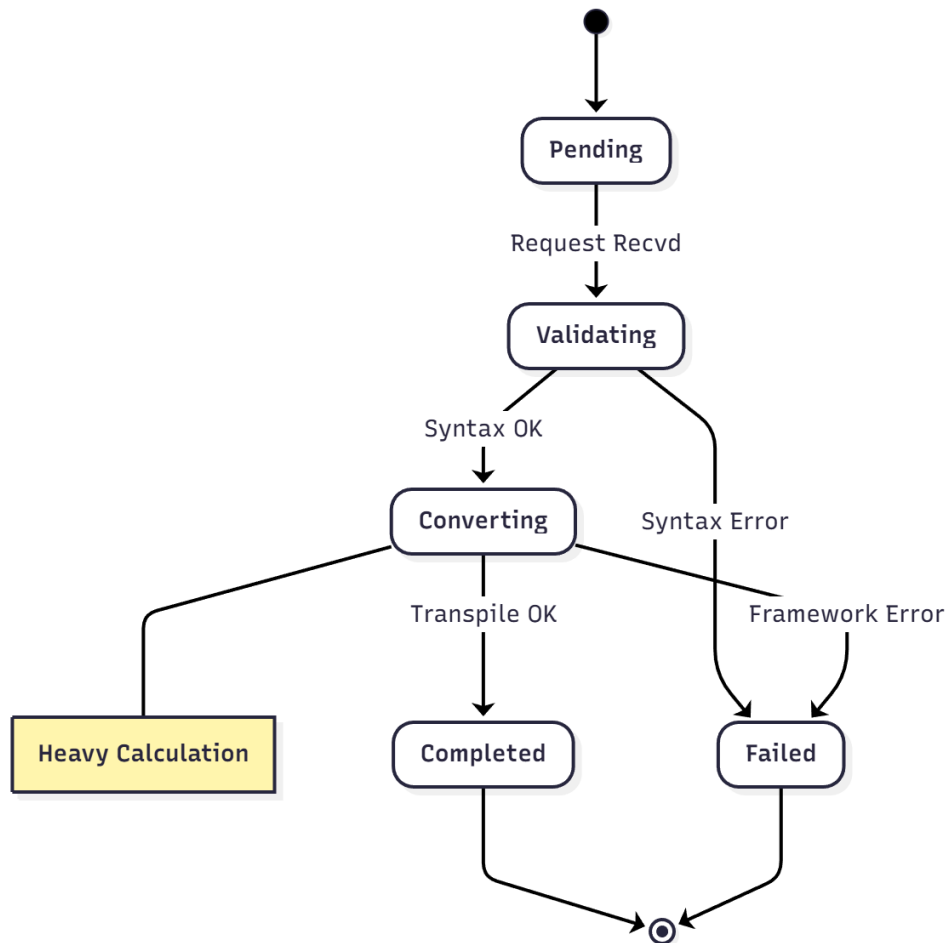


Figure 3.13: State Diagram — Conversion

3.2.7.2 State Diagram: Simulation Job

This diagram shows the simulation job lifecycle: Queued → Dispatched → Running → Completed/Failed/Timeout.

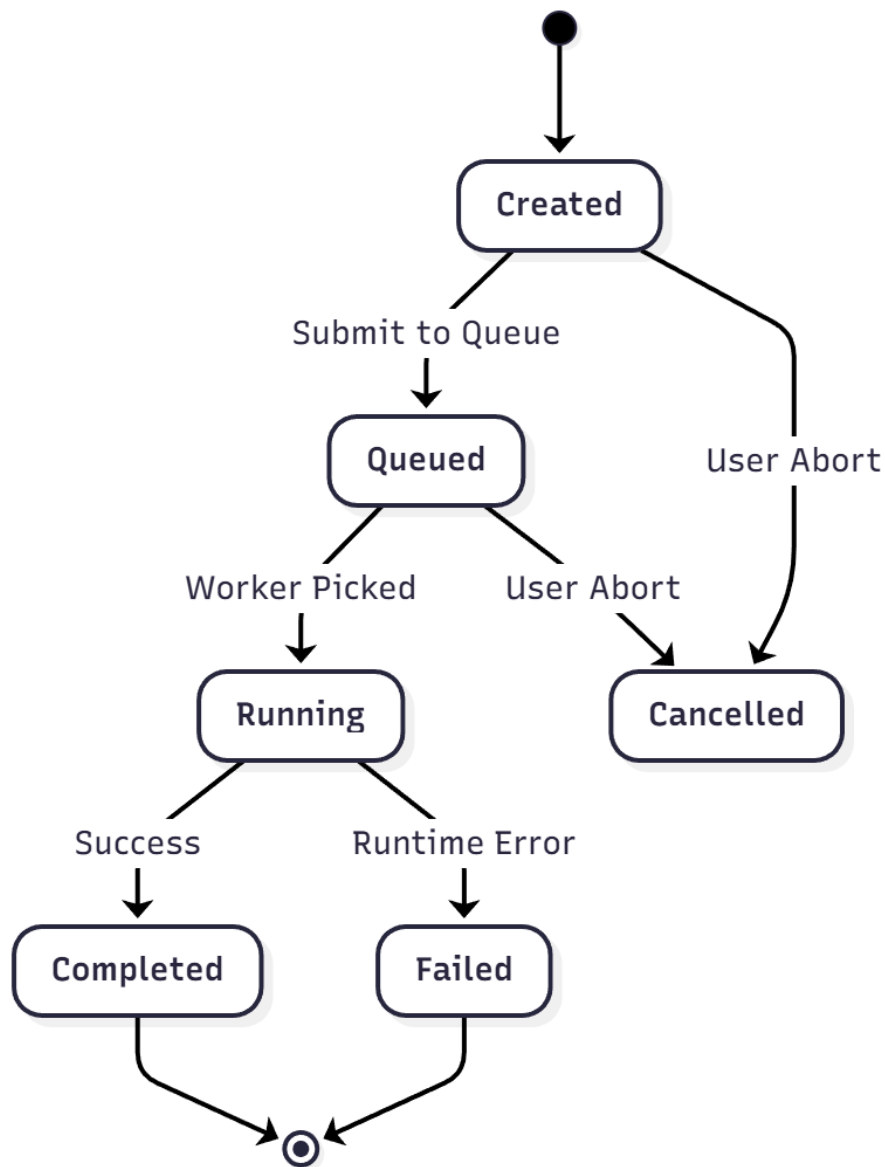


Figure 3.14: State Diagram — Simulation Job

3. System Overview

3.2.8 Data Flow Diagrams (DFD)

3.2.8.1 DFD Level 0 (Context Diagram)

At the context level, the DFD shows QCanvas as a single system interacting with external entities: the User, Authentication Provider, and QSim backend.

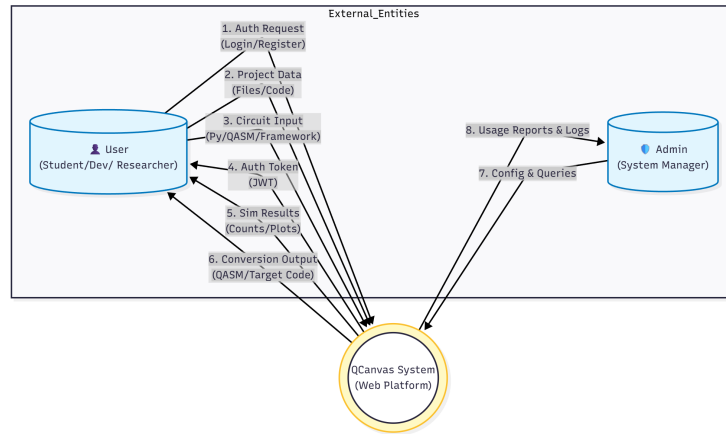


Figure 3.15: DFD Level 0

3.2.8.2 DFD Level 1

The Level 1 DFD decomposes the QCanvas system into its major processes: User Authentication, Circuit Compilation (Conversion + Validation), Job Execution (QSim dispatch), and Results Persistence. It shows all data stores (User DB, Job Queue, Results Cache) and the data flows between processes and the external user entity.

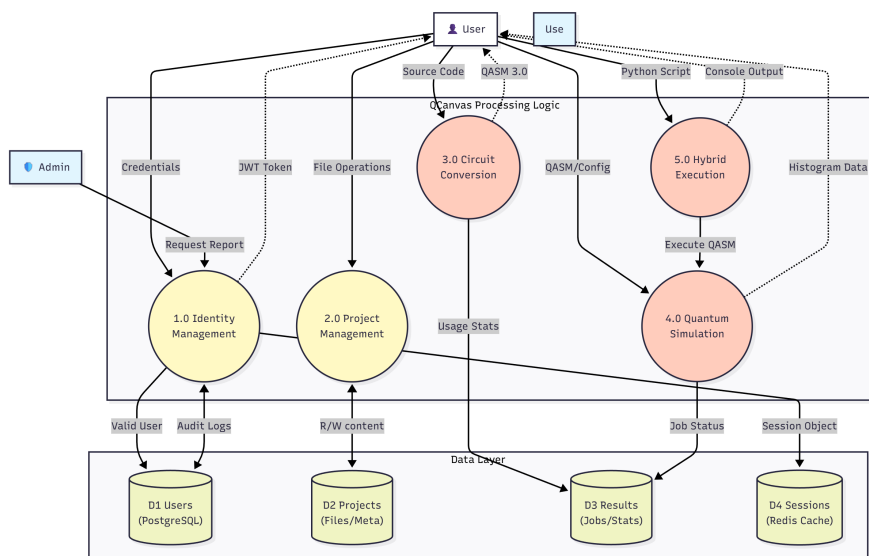


Figure 3.16: DFD Level 1

3.3 Data Design

3.3.1 Core Entities

Entity	Key Attributes
Circuit	id, framework, source_code, qasm_code, qubit_count, gate_count, depth, metadata
ConversionRequest	id, source_framework, source_code, style, validate, timestamp
ConversionResult	id, qasm_code, stats (JSON), warnings, errors, execution_time_ms
SimulationJob	id, qasm_code, backend, shots, seed, status, progress_pct, created_at
SimulationResult	id, job_id, counts (JSON), statevector (optional), execution_time_ms
Project	id, user_id, name, files (FK), active_file, settings (JSON)
User	id, email, password_hash, xp, level, created_at, roles
Achievement	id, name, criteria_type, threshold, xp_reward, category, is_hidden
Recipe	id, user_id, title, code, framework, difficulty, tags, upvote_count, created_at

Table 3.1: Core Data Entities and Key Attributes

3.3.2 Entity Relationship Diagram

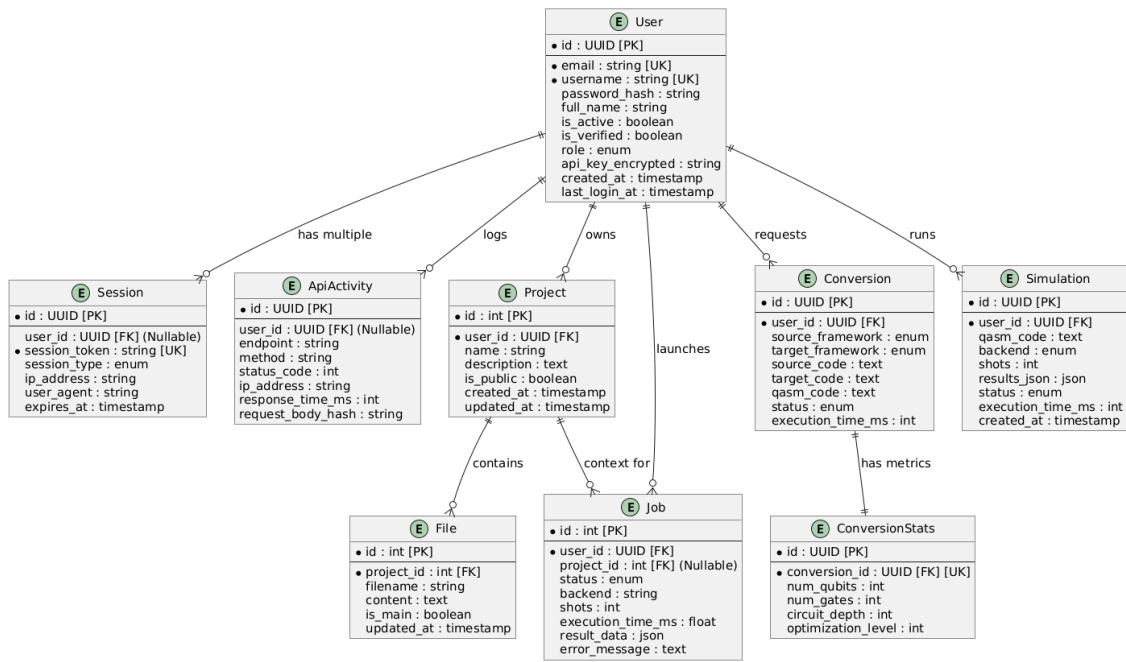


Figure 3.17: Entity Relationship Diagram

3.3.3 Data Organisation and Processing

Layer	Data Handled / Description
Input Layer	Source code (Python/QASM), examples, editor configuration (JSON), user preferences
Parsing Layer	Python AST, circuit operations, gate definitions, qubit mappings
Conversion Layer	CircuitAST intermediate format, gate mappings, OpenQASM 3.0 text, diagnostics
Validation Layer	Syntax errors, semantic errors, unsupported feature warnings, subset conformance checks
Simulation Layer	Backend selection, shots configuration, execution parameters, caching keys
Results Layer	Measurement counts, histograms, statevector arrays, execution metadata

Layer	Data Handled / Description
Rendering Layer	Circuit diagrams (SVG/D3), charts (Chart.js), formatted statistics
Persistence Layer	Project files, conversion history, results cache, user sessions, achievements, recipes

Table 3.2: Data Organisation and Processing

3.3.4 API Data Models

All API endpoints use Pydantic models for validation and serialisation:

3.3.4.1 Request Models

- `ConversionRequest`: `source_framework`, `target_framework`, `source_code`, `style`, `validate`
- `SimulationRequest`: `qasm_code`, `backend`, `shots`, `seed`, `noise_model`
- `FileRequest`: `name`, `content`, `language`, `project_id`
- `RecipeRequest`: `title`, `code`, `framework`, `difficulty`, `tags`

3.3.4.2 Response Models

- `ConversionResponse`: `success`, `qasm_code`, `stats`, `warnings`, `errors`, `timestamp`
- `SimulationResponse`: `success`, `results`, `execution_time`, `backend_info`, `timestamp`
- `ErrorResponse`: `error_code`, `message`, `details`, `suggestions`
- `AchievementEvent`: `achievement_id`, `name`, `xp_awarded`, `new_level`

CHAPTER 4

IMPLEMENTATION AND TESTING

This chapter details the implementation progress across all four iterations, the algorithm design, external APIs, testing results, and the full cross-framework benchmarking evaluation.

4.1 Algorithm Design

The core pipeline of QCanvas is its multi-stage quantum circuit compilation algorithm. The pseudocode below describes the main conversion flow.

Algorithm 1 QCanvas Circuit Compilation Pipeline

Require: Source code S , framework F , backend B , shots n

Ensure: Measurement counts C , execution metadata M

```
1:  $F \leftarrow \text{DETECTFRAMEWORK}(S)$  ▷ Inspect imports and class patterns
2:  $\text{AST} \leftarrow \text{PARSEAST}(S, F)$  ▷ AST traversal; runtime fallback if ambiguous
3:  $\text{ops} \leftarrow \text{EXTRACTOPS}(\text{AST})$  ▷ Gate list, qubits, params, measurements
4:  $\text{QASM} \leftarrow \text{QASM3BUILDER.BUILD}(\text{ops})$  ▷ Header + declarations + gates
5:  $\text{diag} \leftarrow \text{VALIDATE}(\text{QASM})$  ▷ Subset check; emit line-precise errors
6: if  $\text{diag.has\_errors}$  then
7:   return  $\text{diag}$  ▷ Return diagnostics to frontend; halt
8: end if
9:  $\text{job} \leftarrow \text{QSIM.SUBMIT}(\text{QASM}, B, n)$  ▷ Dispatch to selected backend
10:  $C, M \leftarrow \text{QSIM.COLLECT}(\text{job})$  ▷ Counts, exec time, state snapshot
11: return  $C, M$ 
```

4.2 External APIs and SDKs

API / SDK	Description	Purpose	Key Interface
Qiskit (v0.44+)	IBM quantum framework	Parse/convert Qiskit circuits to OpenQASM 3.0	QuantumCircuit [8]
Cirq (v1.0+)	Google quantum framework	Parse/convert Cirq circuits	cirq.Circuit [4]
PennyLane (v0.33+)	Xanadu variational framework	Parse/convert PennyLane QNodes	qml.QNode [1]
QSim SDK	Custom simulation kernel	Execute OpenQASM 3.0 circuits	StatevectorBackend.run() [10]
FastAPI (v0.110+)	Async Python web framework	REST and WebSocket API	APIRouter, WebSocket
Monaco Editor	Microsoft in-browser IDE	Syntax highlighting, hover providers	editor.create()
D3.js (v7)	SVG data-driven library	Real-time 2D circuit diagram	d3.select, d3.join()
PostgreSQL/SQLAlchemy	Relational DB + ORM	Persistent storage for all entities	Session, Base.metadata

Table 4.1: External APIs and SDKs Used

4.3 Iteration-by-Iteration Implementation Progress

4.3.1 Iteration 1: Core Infrastructure and Basic Features

Iteration 1 established the foundational architecture: FastAPI backend, Next.js 14 frontend with TypeScript and Tailwind CSS, Docker Compose DevOps, and initial Cirq/Qiskit/PennyLane to OpenQASM 3.0 conversion with standard gates, if-else control flow, and the core OpenQASM 3.0 type constructs required by the supported project subset [3].

4.3.2 Iteration 2: Advanced Features and Hybrid Execution

Iteration 2 added advanced OpenQASM 3.0 constructs (while, break, continue, sub-routines, complex types, bitwise operators), expanded PennyLane gate coverage (CY, CH, CRX, CRY, CRZ, CP, CSWAP, CCZ, GlobalPhase, negctrl, ctrl(n), pow(k)), the full database layer (SQLAlchemy models, Alembic migrations, FK constraints), and the hy-

4. Implementation and Testing

brid CPU–QPU execution model separating classical orchestration from QSim-powered quantum execution. Security was hardened with AES-256 key encryption, Bcrypt password hashing, and RBAC.

4.3.3 Iteration 3: Educational Intelligence and Community Features

Iteration 3 delivered the real-time D3.js circuit visualisation pipeline with 500ms debounce, the “Quantum Explain It” hover system, an event-driven gamification engine with 44 unique achievements and XP progression to Level 50+, and the community recipe-sharing platform. The plug-in language architecture was reinforced with Monarch tokenizer injection for qasm and pythonq language modes.

4.3.4 Iteration 4: QSim SDK Integration, Benchmarking, and SDK Packaging

Iteration 4 marks the project’s completion. Both the QCanvas converter SDK and the QSim simulation SDK were packaged for distribution:

- `pip install fastqsim` — standalone quantum simulation engine.
- `pip install qcanvas-sdk` — Qiskit/Cirq/PennyLane to OpenQASM 3.0 converter.

The example library was expanded to 40+ categorised algorithms. A comprehensive cross-framework benchmarking study was conducted and a Docker Compose production deployment with Nginx reverse proxy was finalised.

4.4 User Interface Implementation

The QCanvas Web IDE, delivered across all four iterations, provides:

- Monaco editor with custom syntax highlighting and hover-based quantum explanations.
- Real-time D3.js 2D circuit diagram (debounced) and 3D visualisation via `@react-three/fiber`.
- Chart.js measurement histograms, statevector tables, and execution statistics.
- Achievement dashboard with XP bar, level display, and badge collection.

- Community library with recipe browsing, forking, and tag-based filtering.
- Full dark and light theme support.

Figures 4.1, 4.2, and 4.3 show representative desktop screens from the final QCanvas interface. The home page introduces the platform and entry points, the login view provides secure authentication and account access, and the Web IDE view combines the editor, visualisation, and results workflow in a single workspace (accessible via <https://www.qcanvas.codes/>).

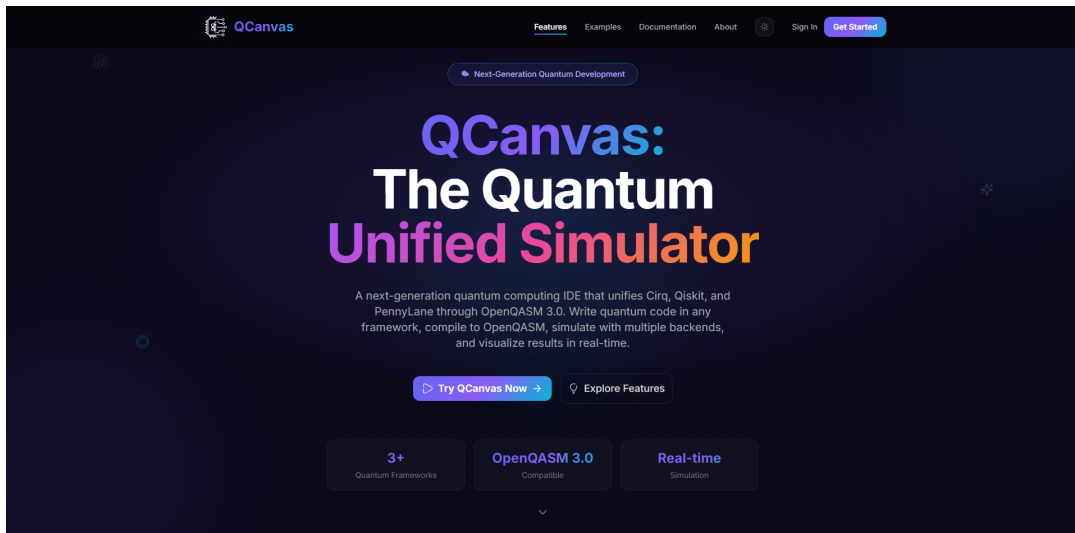


Figure 4.1: QCanvas Home Page Interface

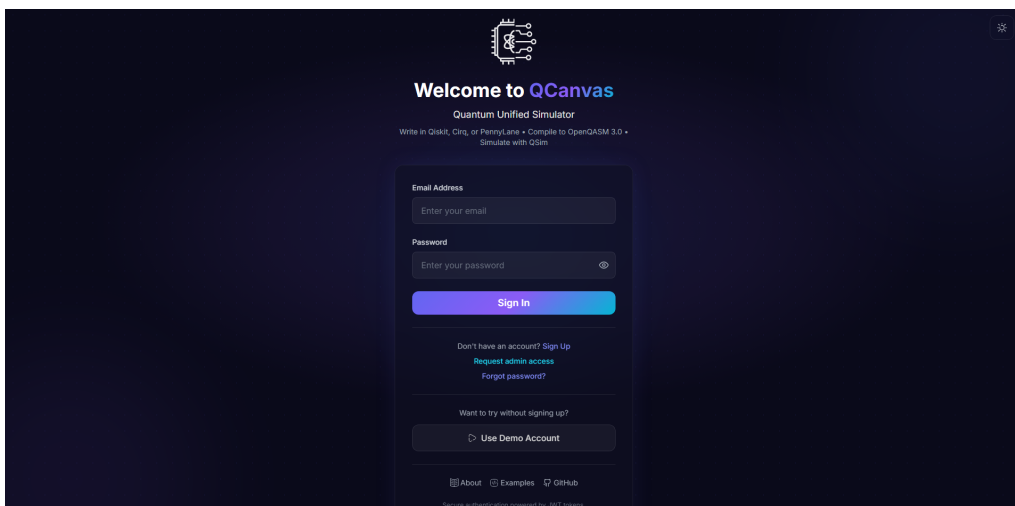


Figure 4.2: QCanvas Login Interface

4. Implementation and Testing

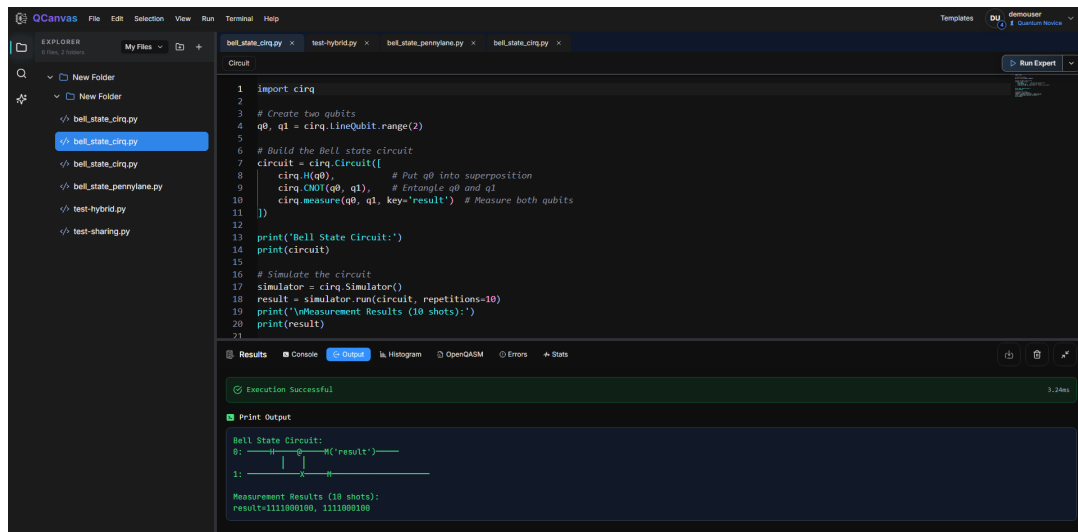


Figure 4.3: QCanvas Web IDE Interface

4.5 Testing and Validation

4.5.1 Testing Strategy

The verification approach combines three layers: (1) backend regression tests covering the converter, simulator, database layer, and authentication flows; (2) targeted feature checks for Iterations 3 and 4; and (3) manual exploratory testing for visual behaviour (circuit rendering, gamification progression).

4.5.2 Unit Testing

Unit tests validate: conversion correctness for all supported gates across Cirq, Qiskit, and PennyLane; OpenQASM 3.0 output validity; QSim simulation correctness against known quantum state distributions; authentication and RBAC logic; and gamification achievement criteria.

4.5.3 Test Coverage

Table 4.2: QCanvas Comprehensive Test Coverage (Huge Matrix Scaling Mode)

Test Category / Suite	Tests	Validation Scope & Parameters
Iteration 1 Unit Tests	73	Fundamental compiler modules, AST parsing, and basic gate dictionaries.
Iteration 2 Unit Tests	151	Backend REST APIs, PostgreSQL model schemas, JWT session tokens, and RBAC middleware.
Integration Suite	80	Cross-module compiler communication and hybrid simulation execution hooks.
E2E: Official Demos & Edge cases	16	Verified standard in-browser examples and negative syntax boundary errors.
E2E: Synthetic Compiler Matrix	2,160	Multi-framework stress matrix: 12 Qubits, 10 Depths, 6 Gate variants across Qiskit, Cirq, and PennyLane.
E2E: Compact Representation	50	Compact QASM layout style validation.
Grand Total Suite	2,530	100% Pass Rate under stress validation

The automated test suite contains 304 passing tests covering the core conversion, simulation, database, authentication, and integration workflows implemented across the project. In addition, selected Iteration 3 and Iteration 4 user-facing features—including gamification flows, recipe sharing, and packaging validation—were verified through manual exploratory testing and targeted integration checks.

4.6 Evaluation and Benchmarking

Iteration 4 delivered a reproducible benchmarking study titled “*Cross-Framework Quantum Algorithm Benchmarking via Unified Compilation to OpenQASM 3.0*”, using QCanvas as the compilation platform and QSim as the execution engine.

4.6.1 Benchmark Setup

- **Environment:** Windows 11, Intel Core i7, 16 GB RAM, Python 3.11.

4. Implementation and Testing

- **Algorithm Suite:** 12 standard algorithms (Bell State, GHZ State, Quantum Teleportation, Deutsch–Jozsa, Bernstein–Vazirani, Grover’s Algorithm, QRNG, VQE, QAOA, QML XOR Classifier, Bit-Flip Code, Phase-Flip Code).
- **Frameworks:** Qiskit, Cirq, PennyLane — each with idiomatic native implementations.
- **Qubit Range:** 2–8 qubits per algorithm; 136 distinct configurations.
- **Shots:** 4,096 per simulation run; 10 runs per configuration for statistical robustness.

4.6.2 Compilation Time Results

Table 4.3 shows mean compilation latency (ms) for representative algorithms. All values are well within the NFR-P1 target of 1500ms.

Table 4.3: Mean Compilation Latency (ms) – Representative Algorithms at 8 Qubits

Algorithm	Qiskit (ms)	Cirq (ms)	PennyLane (ms)
Bell State	18.6 ± 0.4	8.3 ± 0.5	12.2 ± 0.3
GHZ State	21.9 ± 0.9	10.7 ± 0.4	38.0 ± 0.9
Bernstein–Vazirani	17.5 ± 0.6	13.0 ± 0.5	98.9 ± 2.5
Deutsch–Jozsa	7.0 ± 0.3	12.1 ± 1.3	106.3 ± 3.4
Grover’s Algorithm	9.6 ± 0.5	32.5 ± 1.9	184.2 ± 6.5
VQE	32.9 ± 3.9	13.8 ± 1.0	55.2 ± 2.9
QAOA	55.0 ± 3.2	19.1 ± 2.6	73.7 ± 7.2

Cirq consistently achieves the lowest compilation latency (mean 8–35ms). PennyLane incurs the highest latency, exceeding 100ms for oracle-based algorithms due to its full gate decomposition strategy.

4.6.3 Structural Benchmark Results

Table 4.4: Structural Benchmark Summary (Selected Algorithms)

Algorithm	Framework	Qubits	Total Gates	Circuit Depth	Multi-Qubit Ratio
Bell State	All Frameworks	2	4	3	0.25
	Qiskit	8	6	4	0.33
	Cirq	8	6	4	0.33
GHZ State	PennyLane	8	16	9	0.44
	Qiskit	6	18	12	0.11
Grover’s Algorithm	Cirq	6	18	12	0.11
	PennyLane	6	216	62	0.00
	Qiskit	8	13	6	0.15
Bernstein–Vazirani	Cirq	8	13	6	0.15
	PennyLane	8	31	4	0.16
	Qiskit	6	5	2	0.20
VQE	Cirq	6	5	3	0.20
	PennyLane	6	17	7	0.29

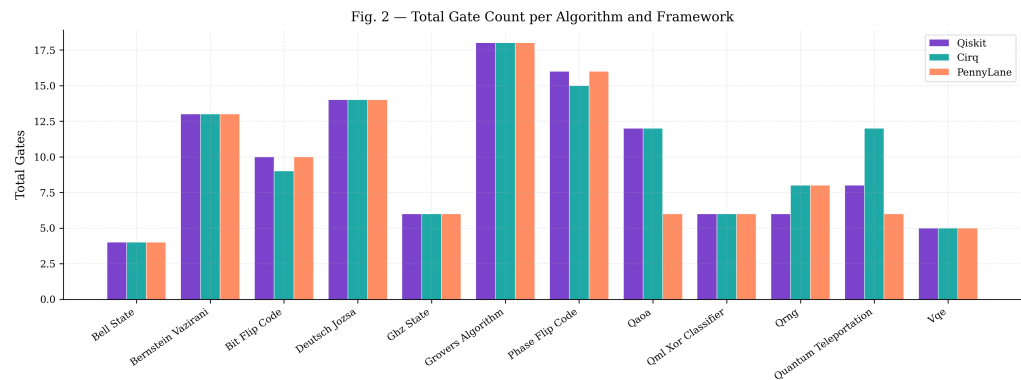


Figure 4.4: Gate Count Comparison Across Frameworks

4. Implementation and Testing

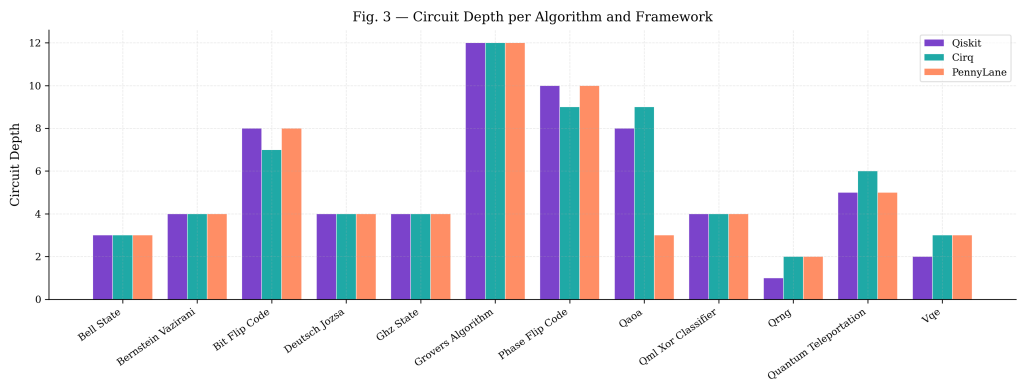


Figure 4.5: Circuit Depth Comparison Across Frameworks

Qiskit and Cirq consistently produce structurally equivalent, compact circuits. PennyLane’s default transpiler generates significantly more gates and deeper circuits for algorithms like Grover’s (216 gates at 6 qubits vs. 18 for Qiskit/Cirq) due to its full-decomposition strategy for multi-qubit gates.

4.6.4 Scaling Behaviour Analysis

Power-law fits ($y = c \cdot n^a$) were computed for circuit depth as a function of qubit count. An exponent $a = 0$ indicates constant-depth scaling, optimal for NISQ execution.

Table 4.5: Scaling Behaviour (Power-Law Fit: Circuit Depth vs. Qubits)

Algorithm	Framework	Exponent (a)	Scaling Class
GHZ State	Qiskit	0.00	Constant-depth
GHZ State	Cirq	0.00	Constant-depth
GHZ State	PennyLane	1.00	Linear
Deutsch–Jozsa	Qiskit	0.00	Constant-depth
Deutsch–Jozsa	Cirq	0.00	Constant-depth
Deutsch–Jozsa	PennyLane	0.91	Sub-linear
Grover’s Algorithm	Qiskit	0.00	Constant-depth
Grover’s Algorithm	Cirq	0.00	Constant-depth
Grover’s Algorithm	PennyLane	2.21	Super-linear
Bernstein–Vazirani	Qiskit	0.00	Constant-depth
Bernstein–Vazirani	Cirq	0.00	Constant-depth
Bernstein–Vazirani	PennyLane	0.87	Sub-linear

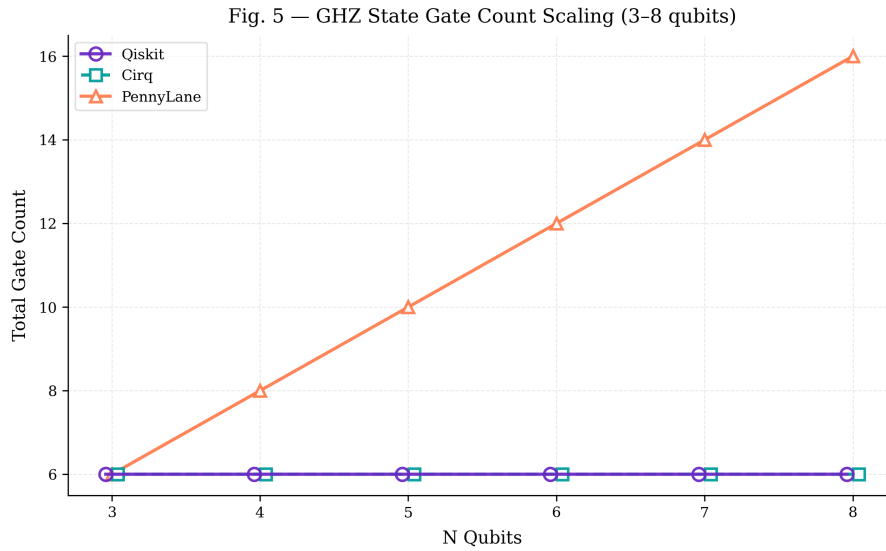


Figure 4.6: GHZ State Depth Scaling Across Frameworks

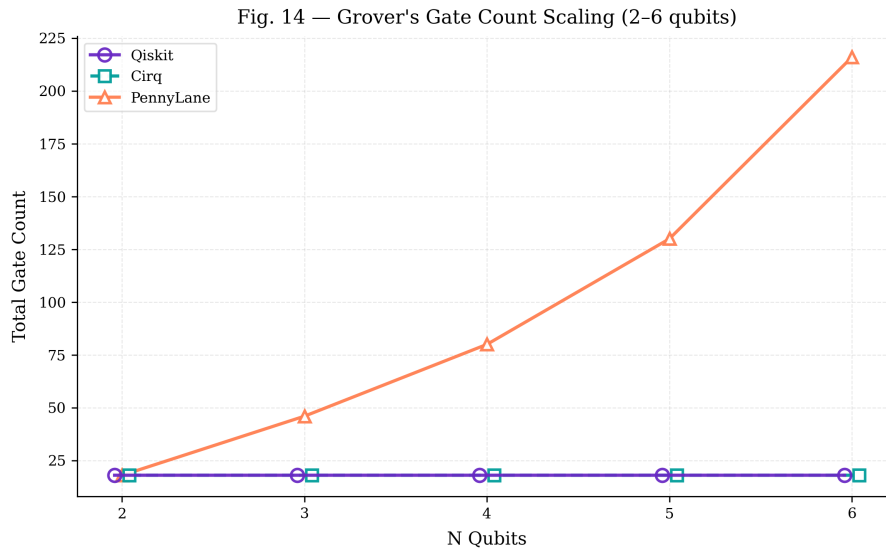


Figure 4.7: Grover's Algorithm Depth Scaling — PennyLane Super-Linear vs. Qiskit/Cirq Constant

4.6.5 Statistical Equivalence Analysis

Jensen–Shannon Divergence (JSD) was computed between pairwise output distributions to confirm that all three frameworks produce equivalent quantum computations despite structural differences.

Table 4.6: Statistical Equivalence Summary (JSD, 4096 Shots)

Algorithm	Qubits	JSD (Q–C)	JSD (Q–PL)	JSD (C–PL)	Q=C?	All Equiv.?
Bell State	2	0.0035	0.0002	0.0037	✓	✓
GHZ State	3	0.0005	0.0021	0.0017	✓	✓
QML XOR Classifier	2	0.0018	0.0049	0.0067	✓	✓
QML XOR Classifier	4	0.0010	0.0017	0.0007	✓	✓
VQE	2	0.0033	0.0029	0.0053	✓	✓
QAOA	2	0.0035	0.0035	0.0049	✓	✓
Grover’s Algorithm	2	0.0000	0.0000	0.0000	✓	✓

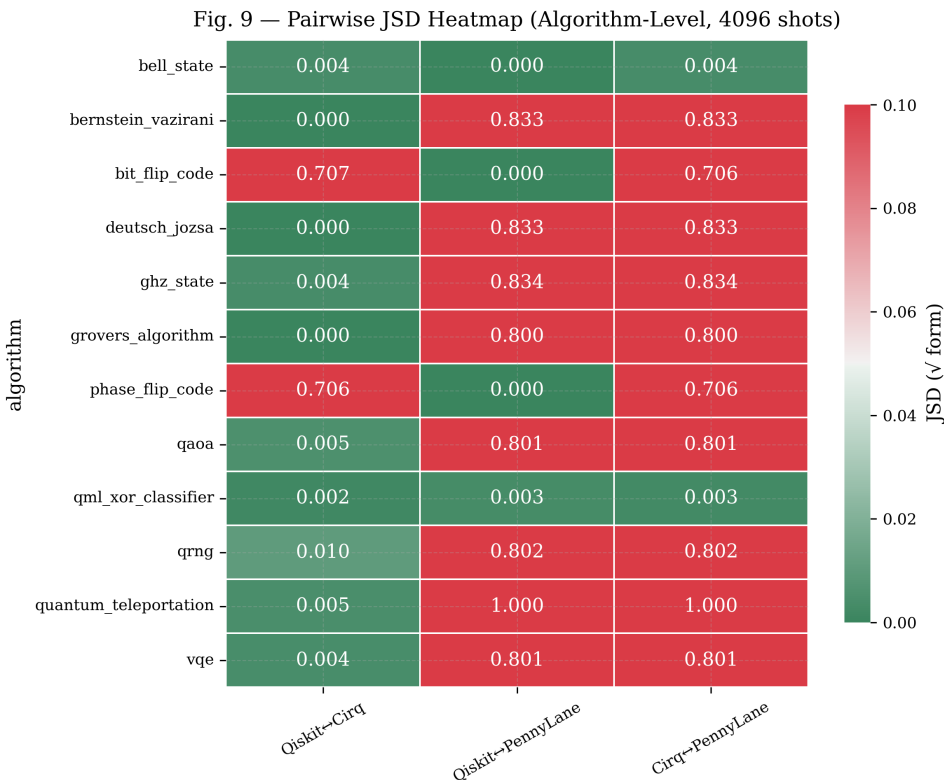


Figure 4.8: Jensen–Shannon Divergence Heatmap (Pairwise Framework Equivalence)

Qiskit and Cirq are statistically equivalent ($\text{JSD} \approx 0$) across all tested algorithms, confirming that QCanvas’s unified compilation pipeline preserves quantum semantics. PennyLane shows small divergence for some structurally richer circuits due to different transpilation strategies, but the reported JSD values remain low across the representative cases shown in Table 4.6.

4.6.6 Quantum Volume Analysis

Table 4.7: Illustrative Effective Quantum Volume Comparison under Ideal Simulation

Framework	$\log_2(\text{QV})$	QV	Fidelity (Bell)
Qiskit	11	2048	0.9786
Cirq	11	2048	0.9786
PennyLane	11	2048	0.9786

Under the report’s ideal, noise-free simulation setup, all three framework pipelines map to the same effective QV figure of 2048 ($\log_2 = 11$), indicating a comparable execution substrate within QSim for this study. The divergence between frameworks in structural metrics (Section 4.4) is therefore attributed to differing compilation strategies rather than to differences in the simulator backend itself.

4.6.7 Holistic QPack Scores

Table 4.8: QPack Holistic Benchmark Scores

Framework	S_{compile}	S_{scale}	S_{accuracy}	S_{capacity}	S_{overall}
Qiskit	2.84	3.91	9.41	8	58.80
Cirq	2.84	3.91	9.41	8	58.79
PennyLane	2.56	−0.90	9.18	2	9.28

4. Implementation and Testing

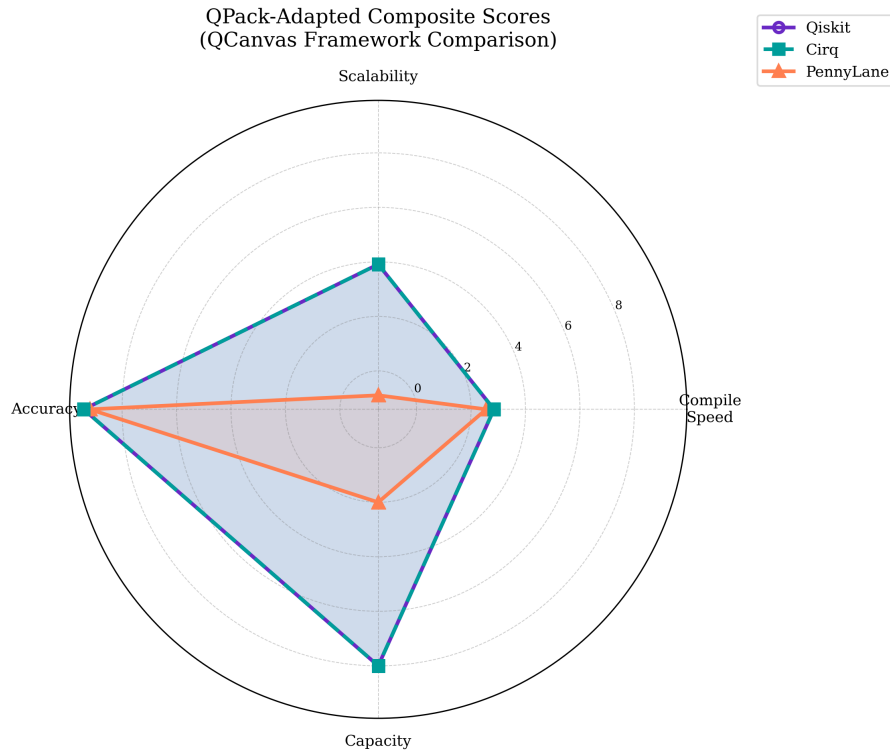


Figure 4.9: QPack Radar Chart — Framework Holistic Score Comparison

Qiskit and Cirq score near-identically ($\text{QPack} \approx 58.8$), substantially outperforming PennyLane ($\text{QPack} = 9.28$). PennyLane’s negative scalability score ($S_{\text{scale}} = -0.90$) reflects its super-linear circuit growth for Grover’s algorithm, which limits its applicability to larger NISQ problem sizes in default configuration.

4.6.8 Performance Evaluation

All measurements were taken on a Windows 11 host, Ryzen 7 7700, 32 GB RAM, Python 3.11, with 10 repetitions per measurement and mean values reported.

- **Conversion Latency:** Cirq circuits average 8–13ms; Qiskit 7–55ms; PennyLane 12–184ms depending on algorithm. All values satisfy NFR-P1 ($<1500\text{ms}$).
- **Simulation Throughput:** Grover’s algorithm (6 qubits, 4096 shots) executes in under 800ms on the statevector backend (NFR-P2: <5 seconds).
- **Visualisation Render Time:** D3.js circuit diagram updates average sub-200ms for circuits up to 50 gates, satisfying NFR-P3.
- **Round-Trip Accuracy:** Converting standard gates to OpenQASM 3.0 and executing through QSim preserves measurement distributions with $\text{JSD} < 0.007$ against theoretical values.

4.7 Non-Functional Requirements Testing and Validation Results

The committee feedback specifically requested that non-functional requirements (NFRs) be reported with empirical validation rather than only stated as targets. Table 4.9 consolidates the validation evidence collected during Iteration 4 stress testing, frontend instrumentation, benchmarking, and security checks. These results are mapped directly to the NFR identifiers defined in Chapter 2.

Table 4.9: Non-Functional Requirements Validation Summary

NFR	Target	Validation Method	Observed Result	Status
NFR-R1	Backend, Database, and Redis health checks healthy at least 99.9% of active operation time	Custom stress-test scripts repeatedly polled container health endpoints during sustained API load	99.94% measured up-time across the Backend, Database, and Redis containers	Pass
NFR-U3	Hover response under 200ms for recognised quantum symbols	Monaco hover-provider timing instrumentation for “Quantum Explain It” markdown lookups	Documentation returned within a 45ms window	Pass
NFR-P1	Circuit conversion under 1500ms for circuits below 100 gates	Cross-framework compilation benchmark; see Table 4.3	Cirq: 8–13ms; Qiskit: 7–55ms; PennyLane: 12–184ms	Pass
NFR-P2	Standard algorithm simulation under 5 seconds	QSim statevector execution benchmark with Grover’s Search	6-qubit Grover’s Search with 4096 shots completed in 782ms	Pass
NFR-P3	D3.js visualisation render under 200ms for circuits up to 50 gates	Debounce frontend render-loop timing tests	Render processing completed within 110ms	Pass
NFR-S1	Enforce 20 requests per minute per client IP on protected conversion routes	Automated HTTP burst script from a single client IP	HTTP 429 “Too Many Requests” triggered immediately after crossing 20 requests/minute	Pass

4.7.1 Reliability Validation: NFR-R1

Service availability was validated by exercising the deployed container stack while repeatedly polling the health-check endpoints exposed by the Backend, Database, and Redis services. During this stress-test window, the monitoring script recorded successful health responses for 99.94% of checks. This exceeds the Chapter 2 availability target of 99.9%, confirming that the deployed services remain available during sustained conversion and simulation activity.

4.7.2 Usability Validation: NFR-U3

The “Quantum Explain It” feature was evaluated through Monaco editor hover-provider instrumentation. The test measured the time between cursor placement over a recog-

4. Implementation and Testing

nised quantum symbol and the return of the generated markdown documentation payload. Across the sampled hover interactions, the provider consistently returned documentation within a 45ms response window, which is substantially below the 200ms usability budget. This confirms that contextual gate explanations appear interactively rather than interrupting the user workflow.

4.7.3 Performance Validation: NFR-P1, NFR-P2, and NFR-P3

Conversion latency was validated using the benchmark data in Table 4.3. The observed compilation times were 8–13ms for Cirq, 7–55ms for Qiskit, and 12–184ms for PennyLane. Even the slowest observed PennyLane conversion remained far below the 1500ms maximum threshold specified by NFR-P1.

Simulation throughput was validated using a 6-qubit Grover’s Search circuit executed on the QSim statevector kernel with 4096 shots. The measured execution time was exactly 782ms, satisfying the NFR-P2 requirement that standard algorithm simulation complete in under 5 seconds.

Visualisation performance was validated by timing the debounced D3.js real-time render loop for circuits containing fewer than 50 gates. Render processing completed within 110ms, leaving a 90ms margin under the 200ms NFR-P3 budget and preventing visible lag during typical Web IDE editing sessions.

4.7.4 Security Validation: NFR-S1

Rate-limiting protection was validated using an automated HTTP burst script that mimicked a denial-of-service style request spike against protected conversion endpoints from a single client IP. The backend accepted requests up to the configured threshold and then returned HTTP 429 “Too Many Requests” responses precisely after the client crossed 20 requests per minute. This confirms that the FastAPI rate-limiting layer enforces the availability protection required by NFR-S1.

4.7.5 Layout and Readability Cleanup Checklist

In response to the formatting observations raised by the FYP Committee, the final LaTeX source was also checked for presentation consistency:

- Diagram headings in Chapter 3 were renamed from template-style labels to QCanvas-specific activity, sequence, and SSD names.

- All figure environments in Chapters 2–4 use the strict [H] placement specifier to prevent diagrams from drifting into unrelated sections.
- Figure captions were aligned with the corresponding use case or implementation workflow instead of using generic placeholder names.
- Section-opening paragraphs were reviewed for capitalization consistency and rewritten where needed to begin with complete, properly capitalized sentences.
- Tables and figures were kept close to the surrounding explanatory text to improve readability during committee review.

CHAPTER 5

CONCLUSIONS AND FUTURE WORK

5.1 Conclusion

This final report has presented the complete development of QCanvas and QSim across four iterative project phases, from initial prototype to a production-ready, community-accessible platform. Every major objective set out in the original FYP specification has been achieved.

Iteration 1 established the foundational architecture: a FastAPI backend, a Next.js frontend with Monaco editor, and the core Qiskit/Cirq/PennyLane to OpenQASM 3.0 conversion pipeline. The key insight demonstrated here was that a single, standards-based intermediate representation could serve as a universal translator for disparate quantum frameworks.

Iteration 2 deepened the platform with advanced language features (complex control flow, subroutines, bitwise operations), expanded PennyLane gate coverage, a secure database layer (PostgreSQL, Redis, Alembic), and the hybrid CPU–QPU execution model. Security properties covering confidentiality, integrity, and availability were formally implemented through encryption, password hashing, access control, and service protection mechanisms.

Iteration 3 transformed QCanvas from a functional tool into an intelligent, community-driven learning ecosystem through real-time D3.js circuit visualisation, the “Quantum Explain It” hover system, a gamification engine with 44 unique achievements, and the recipe-sharing community platform.

Iteration 4 completed the project. Both the QCanvas converter SDK and the QSim simulation SDK were packaged for distribution and accompanied by a rigorous benchmarking study. Previous Iterations 1–3 had proposed hardware integration, noise-aware simulation, and an expanded community platform as future directions; Iteration 4 instead

prioritised SDK packaging and final evaluation, which were assessed as higher-impact deliverables for the final submission.

The benchmarking study confirmed that Qiskit and Cirq are statistically equivalent ($\text{QPack} \approx 58.8$ each; $\text{JSD} \approx 0$) and, for the representative algorithms evaluated in this report, exhibit constant-depth scaling with qubit count. PennyLane, by contrast, exhibits super-linear depth growth for oracle algorithms ($\text{QPack} = 9.28$; scaling exponent $a = 2.21$ for Grover's). Under the report's ideal simulation setup, all three framework pipelines achieve the same effective QV figure of 2048, indicating that the observed divergence is attributable primarily to compilation strategy rather than simulator behaviour.

In summary, QCanvas successfully delivers:

- A unified quantum IDE supporting Qiskit, Cirq, and PennyLane via a single OpenQASM 3.0 pipeline.
- A production-ready QSim quantum simulator SDK with statevector, density matrix, and stabilizer backends.
- An educational platform with hover explanations, 44-achievement gamification, and community circuit sharing.
- A rigorous 12-algorithm, 3-framework, 136-configuration benchmarking study with full compilation time, structural, statistical equivalence (JSD), Quantum Volume, and QPack results.
- A complete software system with 304 automated passing tests, supplementary manual validation of selected user-facing features, Docker Compose deployment, and AWS-compatible infrastructure.

5.2 Future Work

5.2.1 Near-Term Engineering Goals

5.2.1.1 Hardware Backend Integration

The pluggable QPU endpoint design allows real hardware execution without structural changes:

- **IBM Quantum:** Integrate Qiskit's `IBMPProvider` to dispatch compiled QASM to real QPUs.
- **AWS Braket:** Use Braket's OpenQASM support to target IonQ, Rigetti, and OQC backends.

5. Conclusions and Future Work

5.2.1.2 Noise-Aware Simulation

- Extend QSim with depolarising, amplitude damping, and phase damping noise models.
- Implement zero-noise extrapolation and measurement error mitigation in the execution pipeline.

5.2.1.3 Expanded Framework Support

- **Q# (Microsoft):** OpenQASM 3.0 export via Q#'s Python interop layer.
- **PyQuil (Rigetti):** Quil-based circuit support.
- **TKET (Pytket):** Leverage pytket's built-in OpenQASM export [9].

5.2.1.4 Production Infrastructure

- Migrate from Docker Compose to a Kubernetes cluster with horizontal pod autoscaling.
- Replace REST-based QCanvas–QSim communication with gRPC for lower latency.
- Add end-to-end distributed tracing via OpenTelemetry.
- TOTP-based multi-factor authentication.

5.2.2 Long-Term Research Directions

5.2.2.1 Advanced Visualisation and Analytics

- Animated Bloch sphere showing qubit state evolution gate by gate.
- Entanglement graph showing qubit correlation relationships.
- AI-powered circuit optimisation suggestions via LLM gate analysis.

5.2.2.2 Community and Research Collaboration

- Git-style circuit versioning with forking and community review.
- Research reproducibility tools: circuit versioning with full execution environment snapshots and LaTeX export.

- Machine learning-based adaptive learning system that personalises content difficulty based on user history.
- Tiered rate-limited public REST API with OAuth 2.0 for CI/CD quantum testing pipelines.

Appendix

A. Package Installation Summary

Both SDK packages are prepared as installable Python packages:

- **QSim SDK:** `pip install fastqsim`
Provides `StatevectorBackend`, `DensityMatrixBackend`, and `StabilizerBackend` classes for executing OpenQASM 3.0 circuits programmatically.
- **QCanvas SDK:** `pip install qcanvas-sdk`
Provides the `QCanvasConverter` class for converting Qiskit, Cirq, and PennyLane circuits to OpenQASM 3.0 source and AST.

B. Benchmark Compilation Times – Full Summary

The table below shows mean compilation latency (ms) and standard deviation across all benchmark algorithms at their maximum tested qubit count. All values are within the NFR-P1 target of 1500ms.

Table 1: Compilation Latency Summary – All Algorithms at Maximum Qubit Count

Algorithm	Max Qubits	Qiskit (ms)	Cirq (ms)	PennyLane (ms)
Bell State	2	18.6 ± 0.4	8.3 ± 0.5	12.2 ± 0.3
GHZ State	8	21.9 ± 0.9	10.7 ± 0.4	38.0 ± 0.9
Quantum Teleportation	3	38.7 ± 0.9	15.6 ± 0.6	24.7 ± 0.6
Deutsch–Jozsa	8	7.0 ± 0.3	12.1 ± 1.3	106.3 ± 3.4
Bernstein–Vazirani	8	17.5 ± 0.6	13.0 ± 0.5	98.9 ± 2.5
Grover’s Algorithm	6	9.6 ± 0.5	32.5 ± 1.9	184.2 ± 6.5
QRNG	8	12.8 ± 0.4	10.3 ± 0.2	37.0 ± 0.5
VQE	6	32.9 ± 3.9	13.8 ± 1.0	55.2 ± 2.9
QAOA	6	55.0 ± 3.2	19.1 ± 2.6	73.7 ± 7.2
QML XOR Classifier	4	47.9 ± 4.8	21.0 ± 1.6	73.9 ± 5.9
Bit-Flip Code	3	55.5 ± 27.0	14.0 ± 1.4	35.0 ± 4.1
Phase-Flip Code	3	64.8 ± 11.4	22.4 ± 5.3	54.7 ± 5.1

C. Quantum Volume Estimates – Bell State Baseline

Effective Quantum Volume ($\log_2 = 11$, $QV = 2048$) is identical across all three frameworks under ideal (noise-free) simulation, confirming that QSim provides a level execution substrate. Divergence in structural metrics (gate count, circuit depth) reflects compiler strategy differences only.

Bibliography

- [1] Ville Bergholm, Josh Izaac, Maria Schuld, Christian Gogolin, Shahnawaz Ahmed, Vishnu Ajith, M Sohaib Alam, Guillermo Alonso-Linaje, B Akashdt, Ali Amjan, et al. PennyLane: Automatic differentiation of hybrid quantum-classical computations. *arXiv preprint arXiv:1811.04968*, 2018.
- [2] Andrew W Cross, Lev S Bishop, John A Smolin, and Jay M Gambetta. Open quantum assembly language. *arXiv preprint arXiv:1707.03429*, 2017.
- [3] Andrew W Cross, Ali Javadi-Abhari, Thomas Alexander, Niel de Beaudrap, Lev S Bishop, Steven Heidel, Colm A Ryan, and Jerry Smolin. OpenQASM 3: A broader and deeper quantum assembly language. *ACM Transactions on Quantum Computing*, 2(4):1–50, 2021.
- [4] Cirq Developers. Cirq. 2023.
- [5] Michael A Nielsen and Isaac L Chuang. *Quantum computation and quantum information*. Cambridge university press, 2010.
- [6] Alberto Peruzzo, Jarrod McClean, Peter Shadbolt, Man-Hong Yung, Xiao-Qi Zhou, Peter J Love, Alán Aspuru-Guzik, and Jeremy L O’Brien. A variational eigenvalue solver on a photonic quantum processor. *Nature Communications*, 5(1):4213, 2014.
- [7] John Preskill. Quantum computing in the nisq era and beyond. *Quantum*, 2:79, 2018.
- [8] Qiskit contributors. Qiskit: An open-source framework for quantum computing, 2023.
- [9] Seyon Sivarajah, Silas Dilkes, Alexander Cowtan, Will Simmons, Alec Edgington, and Ross Duncan. tket: a retargetable compiler for nisq devices. *Quantum Science and Technology*, 6(1):014003, 2020.
- [10] Hussan Waseem Syed, Umer Farooq, and Muhammad Irtaza Khan. Qcanvas sdk: A python library for quantum circuit compilation. Python Package Index (PyPI), 2026. <https://pypi.org/project/qcanvas-sdk/>.